



**Pontificia Universidad
Católica del Ecuador**
Seréis mis testigos

MANABÍ

**PONTIFICIA UNIVERSIDAD CATÓLICA DEL ECUADOR
SEDE MANABÍ**

CARRERA DE SOFTWARE

TRABAJO DE TITULACIÓN

**DESARROLLO DE UNA PLATAFORMA SAAS QUE INTEGRA
MODULOS CON IA PARA LA GESTIÓN INTEGRAL DE PYMES
GASTRONÓMICAS EN PORTOVIEJO**

LÍNEA DE INVESTIGACIÓN

**INGENIERÍA DE SOFTWARE Y SISTEMAS COMPUTACIONALES PARA LA
TRANSFORMACIÓN DIGITAL EMPRESARIAL**

SUBLÍNEA DE INVESTIGACIÓN

**TRANSFORMACIÓN DIGITAL DE PYMES Y SISTEMAS SAAS
PROCESOS EMPRESARIALES**

**PREVIO AL TÍTULO DE
INGENIERO EN SOFTWARE**

AUTORES

**LUIGGI ANDREE ARTEAGA SANTANA
CHRISTIAN ORLANDO SARAGURO ENCALADA**

TUTOR

JOSE NARANJO, M.ENG.

PORTOVIEJO, FEBRERO 2026

Certificación del Tutor

En mi calidad de tutor del trabajo de integración curricular, certifico haber revisado el presente manuscrito de investigación, el mismo que se ajusta a las normas vigentes de la Pontificia Universidad Católica del Ecuador Sede Manabí, cumpliendo la Normativa del Trabajo de Integración Curricular; en consecuencia, es apto para su presentación y sustentación. En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

JOSE NARANJO, M.ENG.

CI: 1003528674

Tutor

2026

Acta de aprobación del Tribunal

El jurado examinador aprueba el presente trabajo de integración curricular en nombre de la Pontificia Universidad Católica del Ecuador, Sede Manabí.

En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

Nombre del revisor 1

CI:0000000000

Lector Revisor1(a)

Nombre del revisor 2

CI:0000000000

Lector Revisor2(a)

JOSE NARANJO, M.ENG

CI: 1003528674

Tutor

2026

Declaración de Originalidad

Este manuscrito no contiene ningún tipo de material que ha sido aceptado para la obtención de un título universitario en otra institución, excepto en forma de información de soporte que ha sido debidamente citada en mi trabajo. Este trabajo es de total responsabilidad del autor, quien declara bajo juramento que ninguna sección de este trabajo de integración curricular infringe los derechos de autor de nadie.

En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

LUIGGI ARTEAGA

C.I.: 1314693597

Autor

CHRISTIAN SARAGURO

C.I.: 1350274484

Autor

Declaración de Derecho del Autor

Autorizo a la Pontificia Universidad Católica del Ecuador a distribuir este manuscrito de investigación en medios físicos y electrónicos con el fin de promover la divulgación de mis resultados a la comunidad científica y a la sociedad en general. Adicionalmente autorizo el uso de los contenidos de esta investigación como bibliografía para fines académicos, por cualquier medio o procedimiento, citando como fuente de información al autor de este trabajo. En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

LUIGGI ARTEAGA

C.I.: 1314693597

Autor

CHRISTIAN SARAGURO

C.I.: 1350274484

Autor

Dedicatoria

Dedico este trabajo, en primer lugar, a Dios, por ser mi fortaleza en los momentos difíciles y por permitirme alcanzar una de las metas más importantes de mi vida.

A mi familia, por su amor incondicional, sacrificio, paciencia y por creer siempre en mí, incluso cuando yo mismo dudaba. Su apoyo constante ha sido el motor que me impulsó a seguir adelante.

A mis padres, por ser ejemplo de esfuerzo, responsabilidad y perseverancia, y por enseñarme que los sueños se alcanzan con trabajo y dedicación.

Finalmente, dedico este logro a todas aquellas personas que con una palabra de aliento, un consejo o un gesto de apoyo hicieron posible la culminación de esta etapa tan significativa en mi vida.

Agradecimientos

Agradezco profundamente a Dios por brindarme la fortaleza, salud y sabiduría necesarias para culminar esta importante etapa de mi formación profesional.

Mi sincero agradecimiento a mi familia, quienes han sido mi mayor apoyo y motivación constante, brindándome su amor, comprensión y respaldo incondicional en cada paso de este proceso.

Agradezco de manera especial a mis docentes, quienes con su conocimiento, paciencia y orientación contribuyeron significativamente a mi desarrollo académico y profesional, permitiéndome adquirir las herramientas necesarias para la realización de este trabajo.

Un agradecimiento particular a mi tutor/a de tesis, por su guía, tiempo y valiosos aportes durante el desarrollo de esta investigación, siendo un pilar fundamental para su culminación exitosa.

Finalmente, agradezco a todas las personas que, de manera directa o indirecta, colaboraron en la realización de este proyecto y formaron parte de este importante logro personal y profesional.

Resumen

Este trabajo presenta el diseño, desarrollo y validación técnica de una plataforma SaaS modular para la gestión operativa de PYMES gastronómicas en Portoviejo. La solución integra backend (NestJS), panel web y app móvil con módulos de pedidos, inventarios, repartidores, vista de cocina y dashboard analítico, además de un chatbot RAG por WhatsApp. Los medios de pago contemplados son efectivo y transferencia. Los requisitos se levantaron mediante literatura y entrevistas; el cumplimiento se verificó con pruebas de integración y escenarios de prueba en staging. La usabilidad se evaluó con SUS en una PYME piloto. Los resultados evidencian viabilidad técnica y usabilidad aceptable en entorno controlado, aportando una alternativa de digitalización alineada con la Agenda de Transformación Digital 2022–2025.

Palabras clave: PYMES, gastronomía, digitalización, SaaS, Portoviejo.

Abstract

This work presents the design, development, and technical validation of a modular SaaS platform for operational management of gastronomic SMEs in Portoviejo. The solution integrates a NestJS backend, web panel, and mobile app with order management, inventory, delivery, kitchen view, and analytics, plus a WhatsApp RAG chatbot. Payment methods are cash and transfer only.

Requirements were collected through literature review and interviews; compliance was verified via integration tests and staged scenarios. Usability was assessed with SUS in one pilot SME. Results show technical feasibility and acceptable usability in a controlled environment, aligned with Ecuador's 2022–2025 Digital Transformation Agenda.

Keywords: SMEs, gastronomy, digitalization, SaaS, Portoviejo.

Tabla de Contenido

Introducción	1
Antecedentes	2
Contexto del sector gastronómico en Portoviejo	2
Composición empresarial y digitalización en Ecuador	2
Barreras para la digitalización de PYMES	3
Políticas nacionales de transformación digital	3
Justificación de una solución SaaS para PYMES gastronómicas	4
Definición del problema	4
Preguntas de investigación	6
Pregunta general	6
Preguntas específicas	7
Objetivos	7
Objetivo general	7
Objetivos específicos	7
Alcance	8
Método	11
Diseño de la Investigación	11

Enfoque y tipo	11
Población y Muestra	12
Población	12
Muestra y criterios de selección	12
Operacionalización de variables	13
Metodología	14
Técnicas e instrumentos de recolección de datos	16
Consideraciones éticas	18
Procedimiento	19
ETAPA 1: Planificación	19
Arquitectura de la Plataforma	22
Arquitectura de la aplicación móvil	22
Arquitectura de la aplicación web	24
Arquitectura del backend	24
Arquitectura de la base de datos	26
Roles, permisos y seguridad (JWT, RBAC)	30
ETAPA 2: Desarrollo	31
Desarrollo de la Plataforma SaaS	31
Desarrollo del frontend móvil (React Native + Expo)	32
Desarrollo del frontend panel web (Next.js + Tailwind CSS)	34
Desarrollo del backend (NestJS, capas, módulos)	35
ETAPA 3: Validación y Despliegue	43

Arquitectura del despliegue	43
Despliegue en staging y validación técnica	43
Resultados	45
Introducción	45
Resultados del diagnóstico de digitalización	46
Caracterización de la PYME piloto	46
Estado de digitalización previo	46
Necesidades operativas identificadas	48
Resultados de la validación de usabilidad y adopción	49
Usabilidad del sistema	49
Desempeño del sistema en escenarios de prueba	51
Resultados técnicos del software	52
Pruebas unitarias y de integración	53
Métricas de disponibilidad y rendimiento	53
Validación de funcionalidades	54
Problemas identificados y soluciones	55
Costo estimado y financiamiento	56
Cronograma de actividades	56
Discusión	58
Objetivo General: La plataforma SaaS integrada y el significado de los hallazgos	58
Análisis de Objetivos Específicos y Contraste Bibliográfico	59

Identificación y documentación de requisitos (Objetivo Específico 1)	59
Diseño de arquitectura técnica de la plataforma (Objetivo Específico 2)	60
Desarrollo de módulos funcionales principales (Objetivo Específico 3)	60
Implementación e integración del chatbot RAG por WhatsApp (Objetivo Específico 4)	61
Verificación del cumplimiento de requisitos funcionales (Objetivo Específico 5) . .	61
Evaluación de usabilidad con escala SUS (Objetivo Específico 6)	61
Limitaciones de la Investigación	62
Conclusiones	64
Referencias bibliográficas	68
Apéndice A	69
Apéndice B	70
Apéndice C	71
Apéndice D	73
Apéndice E	74
Apéndice F	75
Apéndice G	77
Apéndice H	79

Apéndice I	80
Apéndice J	81

Lista de Figuras

1.	<i>Diagrama de flujo del funcionamiento general de la plataforma SaaS</i>	10
2.	<i>Ciclo iterativo de la Investigación Basada en Diseño</i>	15
3.	<i>Arquitectura por capas de la plataforma SaaS</i>	22
4.	<i>Arquitectura de la aplicación móvil</i>	23
5.	<i>Arquitectura de la aplicación web</i>	24
6.	<i>Arquitectura del backend (API REST)</i>	25
7.	<i>Diagrama Entidad-Relación (ER) de la base de datos</i>	30
8.	<i>Flujo de autenticación y autorización (JWT + RBAC)</i>	32
9.	<i>Flujo de la interfaz móvil por rol (Cliente, Repartidor, Cocina)</i>	33
10.	<i>Flujo del panel web administrativo</i>	35
11.	<i>Diagrama de secuencia: flujo completo del proceso de pedido</i>	41
12.	<i>Lógica del módulo de inventarios y control de stock</i>	42
13.	<i>Flujo del módulo de repartidores</i>	42
14.	<i>Arquitectura de despliegue (VPS, Vercel, APK)</i>	43
15.	<i>Cronograma de actividades del proyecto de titulación</i>	57
16.	<i>Diagrama entidad-relación de la base de datos de la plataforma SaaS</i>	69

17.	<i>Arquitectura por capas de la plataforma SaaS</i>	70
18.	<i>Arquitectura de la aplicación móvil</i>	71
19.	<i>Arquitectura del panel web</i>	72
20.	<i>Arquitectura modular del backend (NestJS) planificada</i>	73
21.	<i>Flujo de autenticación y autorización (JWT + RBAC)</i>	74
22.	<i>Flujo principal del frontend móvil para cliente, repartidor y cocina</i>	75
23.	<i>Flujo principal del panel web administrativo</i>	76
24.	<i>Flujo general del proceso de pedido</i>	77
25.	<i>Diagrama del módulo de inventario</i>	78
26.	<i>Diagrama del módulo de repartidores</i>	78
27.	<i>Arquitectura del despliegue de la plataforma</i>	79
28.	<i>Diagrama de red y arquitectura de la plataforma SaaS web y móvil</i>	80

Lista de Tablas

1.	<i>Operacionalización de variables e indicadores de medición</i>	13
2.	<i>Técnicas e instrumentos de recolección de datos</i>	17
3.	<i>Selección tecnológica para la arquitectura de la plataforma SaaS</i>	20
4.	<i>Desempeño en tareas guiadas por rol de usuario</i>	50
5.	<i>Resultados de métricas de desempeño en escenarios de prueba frente a umbrales de éxito</i>	52
6.	<i>Costo estimado del proyecto</i>	56
7.	<i>Tareas evaluadas en pruebas de usabilidad</i>	82

Introducción

En las últimas décadas, la digitalización empresarial ha transformado radicalmente los modelos operativos y competitivos a nivel mundial. La adopción masiva de tecnologías digitales ha permitido a las organizaciones optimizar procesos, reducir costos operativos, mejorar la experiencia del cliente y tomar decisiones basadas en datos en tiempo real. Sectores como el comercio electrónico, la logística, los servicios financieros y la manufactura han experimentado cambios estructurales profundos mediante la implementación de sistemas ERP, plataformas de gestión integrada, automatización de procesos e inteligencia artificial aplicada. Sin embargo, esta transformación digital no ha sido uniforme ni equitativa: mientras que grandes corporaciones disponen de recursos técnicos, económicos y humanos para adoptar soluciones tecnológicas avanzadas, las Pequeñas y Medianas Empresas (PYMES) continúan enfrentando barreras significativas de acceso, costo, capacitación y adaptabilidad tecnológica.

A nivel latinoamericano y particularmente en Ecuador, las PYMES constituyen la columna vertebral del tejido empresarial, representando más del 90 % de las unidades productivas registradas y generando una proporción significativa del empleo nacional. No obstante, diversos estudios e informes institucionales evidencian que la mayoría de estas empresas mantiene niveles de digitalización limitados, especialmente en sectores tradicionales como la gastronomía, donde los procesos operativos continúan dependiendo de gestión manual, canales de comunicación no integrados y ausencia de herramientas automatizadas para la toma de decisiones. Esta brecha digital no solo limita su competitividad frente a empresas tecnificadas o plataformas digitales consolidadas, sino que también afecta su capacidad de adaptación, escalabilidad y sostenibilidad en mercados cada vez más exigentes, donde los consumidores demandan inmediatez, conveniencia, transparencia y experiencias omnicanal.

Frente a este panorama, las soluciones SaaS (Software as a Service) representan una oportunidad estratégica para democratizar el acceso a tecnología empresarial avanzada sin requerir inversiones iniciales elevadas, infraestructura propia o conocimientos técnicos especializados. Bajo esta perspectiva, el presente estudio se centra en evaluar la factibilidad, diseño, desarrollo y validación de una plataforma SaaS integral orientada específicamente a las PYMES gastronómicas de Portoviejo, una ciudad reconocida internacionalmente por la UNESCO como Ciudad Creativa de la Gastronomía, pero cuyos negocios gastronómicos locales aún enfrentan desafíos significativos en su proceso de modernización digital.

Antecedentes

Contexto del sector gastronómico en Portoviejo

Portoviejo fue declarada Ciudad Creativa de la Gastronomía por la UNESCO en 2019, un reconocimiento internacional que posiciona a la gastronomía local como un elemento estratégico tanto cultural como económico. Este distintivo subraya el potencial del sector para impulsar el desarrollo turístico, generar empleo y fortalecer la identidad regional. Las PYMES gastronómicas constituyen el núcleo de este sector, desempeñando un rol fundamental en la preservación de tradiciones culinarias y en la dinamización de la economía local.

Composición empresarial y digitalización en Ecuador

A nivel nacional, el tejido empresarial ecuatoriano está compuesto en gran parte por PYMES. Según datos del sondeo de adopción digital realizado por Movistar Empresas, cerca del 91 % de las PYMES ecuatorianas planea invertir en digitalización Movistar Empresas / Telefónica Ecuador (2024). En el caso de la provincia de Manabí, donde se encuentra Portoviejo, las PYMES también juegan un rol importante en la generación de empleo y el desarrollo local.

Barreras para la digitalización de PYMES

Uno de los desafíos principales para estas empresas es la limitada adopción de tecnologías digitales. Si bien existe un reconocimiento creciente de su importancia, muchas PYMES siguen enfrentando barreras estructurales como costos elevados, falta de conocimientos técnicos, ausencia de capacitación especializada y sistemas tecnológicos poco adaptados a sus necesidades Ayora Recalde et al. (2024) y Durán Mero (2024).

En muchos negocios gastronómicos pequeños, la gestión operativa (por ejemplo, inventarios y pedidos) continúa siendo manual o con herramientas básicas. Esta falta de automatización puede generar errores, desperdicios y dificultades para proyectar la demanda, lo que impacta negativamente en la rentabilidad y eficiencia. Además, el servicio al cliente todavía depende de canales tradicionales como llamadas telefónicas o WhatsApp, limitando la capacidad de ofrecer una experiencia profesional, rápida y escalable.

Por otra parte, el beneficio de adoptar métodos de pago digital y sistemas integrados es significativo para incrementar eficiencia operativa Vistazo (2025). Aunque no existen estudios exactos para todas las PYMES gastronómicas de Portoviejo, análisis de mercado muestran que la adopción de soluciones digitales puede reducir costos operativos y mejorar márgenes de pequeños emprendimientos.

Políticas nacionales de transformación digital

La Agenda de Transformación Digital del Ecuador 2022–2025 Ministerio de Telecomunicaciones y de la Sociedad de la Información (2022) establece una hoja de ruta institucional para impulsar la adopción de tecnologías en distintos sectores, incluyendo las PYMES. Entre sus objetivos está promover infraestructura tecnológica, facilitar la capacitación y crear un entorno favorable para que las empresas pequeñas accedan a soluciones digitales.

Justificación de una solución SaaS para PYMES gastronómicas

Frente a estas problemáticas, una solución SaaS (Software como Servicio) adaptada a PYMES gastronómicas locales emerge como una alternativa viable y estratégica. Un sistema SaaS puede ofrecer módulos integrados para la gestión de pedidos, control de inventarios, analítica empresarial y atención al cliente automatizada (mediante chatbots), sin requerir inversiones grandes en infraestructura o conocimientos técnicos avanzados por parte de los usuarios.

Por lo tanto, desarrollar una plataforma SaaS que responda a las necesidades operativas, técnicas y económicas de las PYMES gastronómicas de Portoviejo constituye una estrategia viable y alineada con las metas institucionales nacionales de digitalización y con las aspiraciones locales de modernización, competitividad y crecimiento sostenible del sector.

Definición del problema

En el sector gastronómico de la ciudad de Portoviejo, las Pequeñas y Medianas Empresas (PYMES) desempeñan un rol fundamental en la economía local, aportando significativamente a la generación de empleo, al dinamismo comercial y a la identidad gastronómica de la región. A pesar de su importancia, este sector enfrenta un proceso de digitalización lento e incompleto, caracterizado por la utilización de métodos tradicionales en la gestión operativa, administrativa y comercial. En un entorno donde la tecnología se ha convertido en un componente esencial para la competitividad y la sostenibilidad, muchas de estas empresas continúan utilizando procesos manuales para administrar pedidos, inventarios y servicios de delivery, dificultando su adaptación a las nuevas exigencias del mercado digital.

El problema principal identificado es la falta de soluciones tecnológicas accesibles, integradas y adaptadas al contexto local, que permitan a las PYMES gastronómicas gestionar de forma eficiente sus operaciones diarias. Actualmente, la mayoría de estos negocios no dispone de herramientas centralizadas para controlar inventarios, administrar pedidos, gestionar repartidores o brindar

atención al cliente mediante sistemas automatizados. Esto genera desorganización, tiempos de respuesta elevados y errores en la toma de pedidos que afectan directamente la calidad del servicio. La ausencia de una plataforma tecnológica integral limita la capacidad de estas empresas para modernizarse y competir frente a cadenas más grandes o plataformas de delivery externas.

Diversos estudios y diagnósticos sobre transformación digital en Ecuador evidencian que las PYMES mantienen un nivel de adopción tecnológica limitado, especialmente en actividades tradicionales como la gastronomía. Aunque representan más del 90% del tejido empresarial del país, informes como la Agenda de Transformación Digital del Ecuador 2022–2025 señalan que estas empresas aún enfrentan barreras significativas para modernizar sus procesos, debido a limitaciones económicas, escasa capacitación técnica y la falta de soluciones accesibles y adaptadas a su escala operativa. En Portoviejo, pese a ser reconocida por la UNESCO como Ciudad Creativa de la Gastronomía en 2019, la mayoría de los negocios gastronómicos continúa operando sin sistemas integrados de gestión para pedidos, inventarios y atención al cliente, lo cual genera procesos manuales, inconsistencias y baja eficiencia operativa. Esta evidencia muestra una brecha marcada entre el potencial gastronómico de la ciudad y el nivel real de digitalización presente en sus pequeñas y medianas empresas, lo que confirma la existencia de un problema estructural que afecta su competitividad y sostenibilidad.

Las causas del problema se relacionan principalmente con:

1. El alto costo de las soluciones tecnológicas disponibles en el mercado, que suelen estar orientadas a empresas medianas o grandes y no se ajustan al presupuesto de las PYMES locales.
2. La falta de capacitación tecnológica del personal, lo cual dificulta la adopción de plataformas complejas.
3. La ausencia de una oferta local de software adaptable, que incluya en una sola plataforma módulos de delivery, inventarios, atención automatizada y analítica.

4. La dependencia de plataformas de terceros, que cobran comisiones elevadas y no permiten gestionar los procesos de manera personalizada.

Estas causas se combinan para generar un rezago tecnológico que limita el crecimiento y la modernización del sector gastronómico.

Si el problema no se soluciona, las PYMES gastronómicas de Portoviejo continuarán enfrentando dificultades operativas que limitan la calidad del servicio y la experiencia del cliente. La falta de digitalización provocará que estos negocios sigan siendo menos competitivos frente a empresas que ya adoptaron herramientas tecnológicas avanzadas. Además, persistirán los errores en la toma de pedidos, las dificultades en el control de inventarios y los tiempos de entrega elevados debido a una gestión manual de repartidores. A largo plazo, esta situación limitará su capacidad para adaptarse a un mercado que demanda rapidez, precisión y experiencia digital.

Abordar este problema es fundamental para demostrar la viabilidad técnica de una plataforma SaaS accesible e integrada para el sector gastronómico local. El desarrollo de una solución modular permitirá evaluar si la digitalización de procesos operativos (gestión de pedidos, inventarios, repartidores y atención automatizada) es técnicamente factible y usable en el contexto de las PYMES de Portoviejo. Además, esta investigación se alinea con los objetivos nacionales de digitalización y con la visión de Portoviejo como Ciudad Creativa de la Gastronomía. Los resultados de esta validación técnica sentarán las bases para futuras investigaciones orientadas a la implementación a mayor escala y la medición de impacto operativo y económico en el sector.

Preguntas de investigación

Pregunta general

¿Cómo diseñar y desarrollar una plataforma SaaS integral para digitalización de PYMES gastronómicas y cómo demostrar su viabilidad técnica y usabilidad mediante validación en entorno controlado?

Preguntas específicas

1. ¿Cuáles son los requisitos funcionales y no funcionales que debe satisfacer una plataforma SaaS orientada a la gestión operativa de PYMES gastronómicas en el contexto de Portoviejo?
2. ¿Qué arquitectura de software y stack tecnológico resultan adecuados para implementar una solución modular, escalable y mantenible que integre gestión de pedidos, inventarios, repartidores y dashboard analítico?
3. ¿Cómo integrar un módulo de inteligencia artificial basado en tecnología RAG para automatizar la atención al cliente mediante WhatsApp?
4. ¿En qué medida la plataforma desarrollada cumple con los requisitos funcionales especificados y los criterios de calidad de software establecidos?
5. ¿Cuál es el nivel de usabilidad percibida de la plataforma según evaluación con usuarios de prueba en entorno controlado?

Objetivos

Objetivo general

Diseñar, desarrollar y evaluar técnicamente una plataforma SaaS modular que integre funcionalidades de gestión de pedidos, inventarios, repartidores y atención automatizada mediante inteligencia artificial, orientada a las necesidades operativas de PYMES gastronómicas de Portoviejo, mediante validación en entorno controlado.

Objetivos específicos

1. Identificar y documentar los requisitos funcionales y no funcionales mediante revisión de literatura, análisis del dominio y entrevistas con actores del sector gastronómico.

2. Diseñar la arquitectura técnica de la plataforma, definiendo la estructura modular, patrones de diseño, modelo de datos y protocolos de seguridad e integración.
3. Desarrollar los módulos funcionales principales: gestión de pedidos con geolocalización, control de inventarios, administración de repartidores, vista de cocina y dashboard analítico con KPIs.
4. Implementar e integrar un chatbot basado en el modelo RAG accesible vía WhatsApp para automatizar respuestas a consultas frecuentes.
5. Verificar el cumplimiento de requisitos funcionales mediante pruebas de integración y validación de flujos críticos en entorno controlado (staging).
6. Evaluar la usabilidad de la plataforma mediante la aplicación de la escala SUS con usuarios de prueba en entorno controlado.

Alcance

QUÉ HACEMOS:

- Diseñar, desarrollar y validar una plataforma SaaS completa (backend NestJS, app móvil React Native, panel web Next.js, BD PostgreSQL).
- Implementar 6 módulos funcionales: gestión de pedidos (con geolocalización), inventarios, repartidores, vista cocina móvil, dashboard analítico (KPIs), chatbot RAG por WhatsApp.
- Desplegar en entorno controlado (staging): backend en VPS Hetzner, web en Vercel, app móvil como APK Android.
- Validar técnicamente con 1 PYME piloto: pruebas de integración, flujos críticos, evaluación SUS de usabilidad.
- Métodos de pago: únicamente efectivo y transferencia bancaria.

QUÉ NO HACEMOS:

- Arquitectura multi-tenant (una instancia por cliente para este piloto).
- Pasarela de pagos integrada, optimización automática de rutas.
- Módulos adicionales (marketing, reseñas, gestión de proveedores).
- Aplicación nativa iOS, infraestructura de alta disponibilidad.
- Soporte post-implementación, plan de escalamiento operacional real.
- Medición de impacto económico o evaluación a escala sectorial.

Objetivo del alcance: Demostrar viabilidad técnica, funcional y usabilidad de una solución SaaS integrada en entorno controlado (staging), sentando bases para futuras investigaciones a mayor escala.

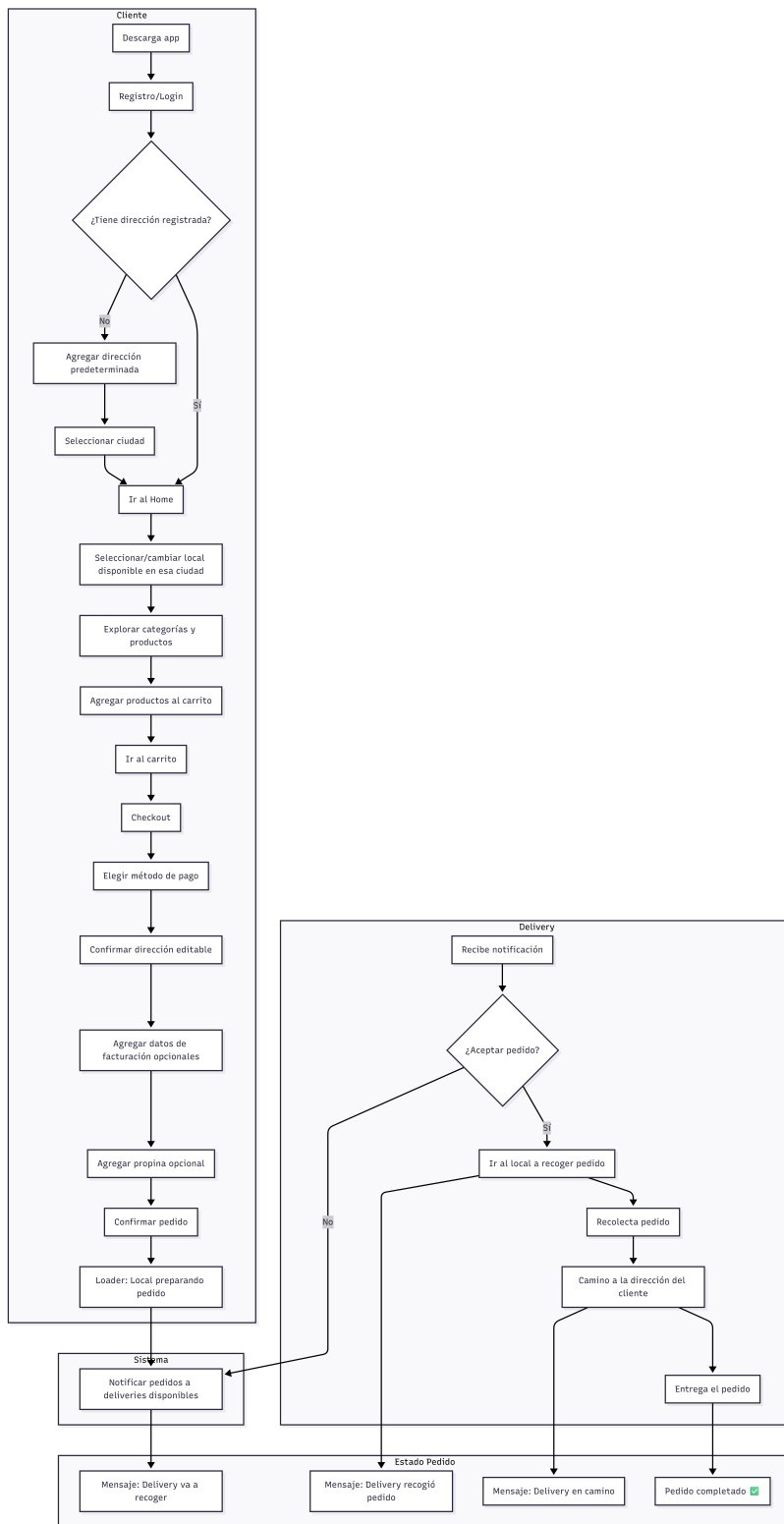


Figura 1. Diagrama de flujo del funcionamiento general de la plataforma SaaS

Método

Diseño de la Investigación

Enfoque y tipo

La investigación se desarrolló bajo un enfoque de Investigación Basada en Diseño (Design Science Research), con evaluación de la solución mediante pruebas técnicas y pruebas de usabilidad en un entorno controlado (staging). Se adoptó un estudio de caso como contexto (PYME piloto), sin medición longitudinal de desempeño operativo real. Este enfoque se eligió para desarrollar y evaluar un artefacto de software (la plataforma SaaS) que resuelve una problemática concreta en un contexto específico.

El componente cualitativo permitió profundizar en las percepciones, necesidades y experiencias de los usuarios durante las pruebas de usabilidad, mientras que el cuantitativo posibilitó la medición objetiva de variables de desempeño en escenarios de prueba controlados. El estudio se clasifica como investigación aplicada de tipo tecnológica, dado que su propósito central es el diseño, construcción y validación técnica de un artefacto de software para demostrar su viabilidad en un contexto específico. El diseño es no experimental y transversal, ya que se observaron y describieron las variables en su contexto de prueba sin manipulación deliberada, en un período específico.

Población y Muestra

Población

La población objetivo estuvo constituida por la totalidad de las Pequeñas y Medianas Empresas (PYMES) del sector gastronómico formalmente establecidas en la ciudad de Portoviejo, provincia de Manabí, Ecuador.

Muestra y criterios de selección

Se aplicó un muestreo no probabilístico por conveniencia y criterio. La muestra final estuvo conformada por una PYME gastronómica. Los criterios de inclusión fueron:

- 1) Ser una PYME ubicada en Portoviejo.
- 2) Contar con servicio de consumo en el local y delivery.
- 3) Manifestar interés en mejorar procesos mediante digitalización.
- 4) Disponer de conectividad básica a internet.
- 5) Otorgar consentimiento para participar en todas las fases del estudio.

Se excluyeron establecimientos con operación irregular o que ya utilizaban sistemas de gestión integral.

Operacionalización de variables

Tabla 1: *Operacionalización de variables e indicadores de medición*

Variable dependiente	Definición operacional	Instrumento de medición	Umbral de éxito
Eficiencia en el registro de pedidos	Tiempo y precisión en la captura y procesamiento de un pedido en tareas guiadas.	Ficha de observación estructurada (registro de tiempos y errores en tareas de prueba).	Tiempo promedio ≤ 3 min por pedido y tasa de errores $\leq 10\%$ en escenarios de prueba.
Eficacia del servicio de delivery	Capacidad del sistema para gestionar y coordinar entregas dentro de un tiempo esperado en escenarios de prueba.	Ficha de observación estructurada (registro de tiempos en flujos de prueba).	Tiempo promedio de entrega ≤ 35 min en escenarios de prueba.
Complejidad de flujos críticos	Capacidad del sistema para ejecutar los flujos principales (pedido, cocina, reparto) sin fallos en tareas guiadas.	Registros de pruebas de integración y observación de flujos (conteo de tareas completadas vs. fallidas).	Tasa de éxito $\geq 95\%$ en la finalización de tareas guiadas.
Estabilidad del sistema	Confiabilidad y continuidad del sistema durante las pruebas, minimizando fallos técnicos.	Registros de logs del sistema y observación directa durante pruebas.	Tasa de fallos técnicos $\leq 5\%$ durante el período de pruebas.
Usabilidad del sistema	Grado de facilidad, eficiencia y satisfacción con que los usuarios pueden emplear la plataforma.	Cuestionario estandarizado System Usability Scale (SUS).	Puntaje promedio ≥ 68 (umbral de aceptabilidad en la escala SUS).

Nota: Los umbrales de éxito para las métricas de desempeño se establecieron con base en estándares razonables de calidad para un sistema en fase de validación técnica, considerando la factibilidad de ejecución en escenarios de prueba controlados. El umbral para el SUS se basa en la clasificación estándar de Bangor et al. (2009).

Metodología

El presente trabajo sigue la metodología Investigación Basada en Diseño (IBD), elegida por ser una metodología de investigación poderosa para desarrollar y evaluar artefactos (productos, procesos, métodos, modelos, etc.) que resuelven problemas del mundo real. No existe un único modelo canónico de fases, ya que distintos autores han propuesto esquemas similares pero con variaciones; sin embargo, todas comparten un núcleo cíclico e iterativo de construcción y evaluación. Se presenta el siguiente esquema integrando los modelos clave de Hevner et al. (2004) y Peffers et al. (2007).

Fase 1. El objetivo de esta fase fue comprender y definir claramente el problema. Se realizó una revisión de la literatura del tema, contextualizándolo en el ámbito global y local.

Fase 2. Una vez comprendido el problema, se establecieron los objetivos que busca conseguir el producto. En esta fase se definieron los requisitos funcionales y no funcionales de la plataforma.

Fase 3 y Fase 4. Forman el ciclo central de construcción-evaluación de la plataforma. Incluye desde la selección de las herramientas tecnológicas, la arquitectura y el desarrollo de la solución. Se trabaja en ciclos cortos.

- Se diseña y desarrolla un incremento (un prototipo, una versión, un módulo).
- Inmediatamente se demuestra y prueba (en un entorno piloto, con usuarios, en simulación).
- Los resultados de esa demostración alimentan directamente el rediseño y desarrollo de la siguiente iteración.

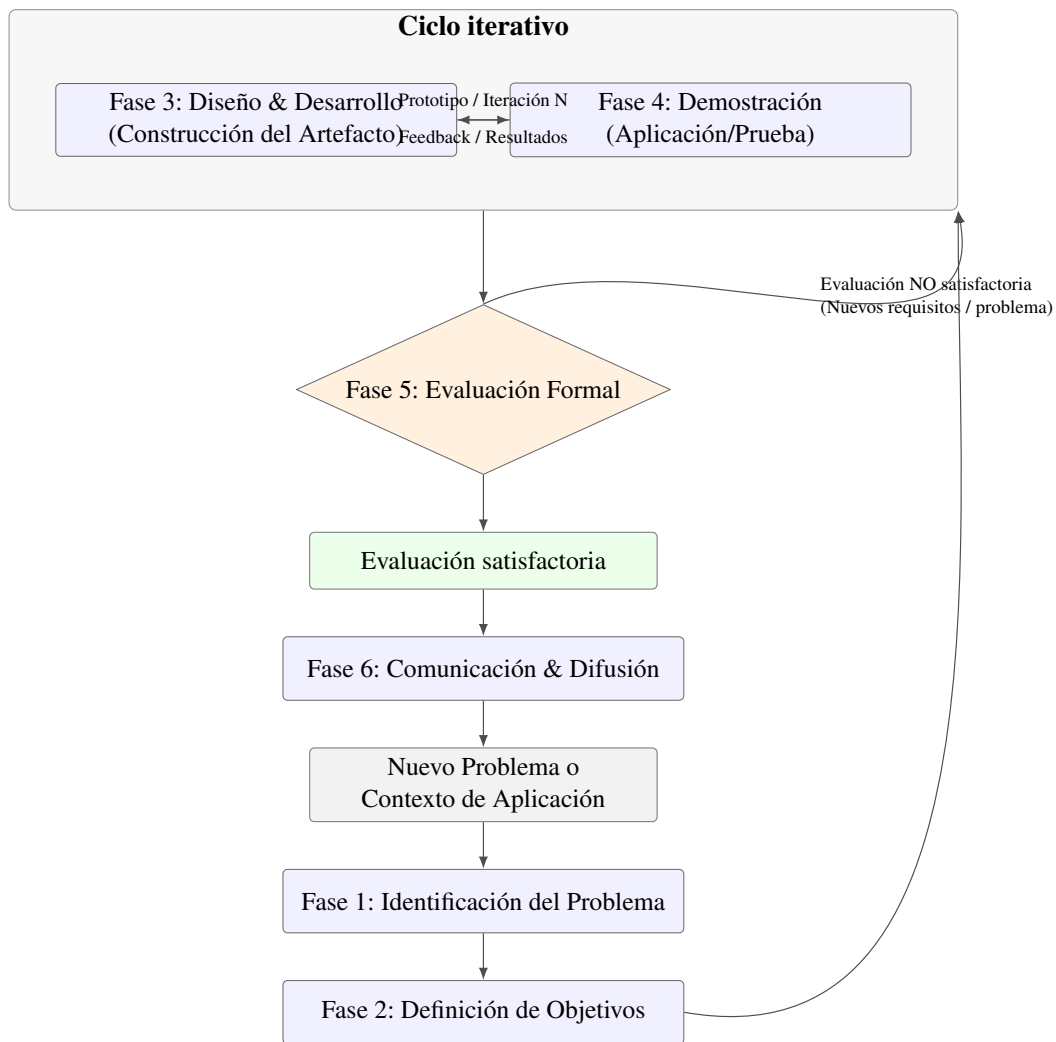


Figura 2. Ciclo iterativo de la Investigación Basada en Diseño

Proceso ágil y sprints

La implementación ágil se estructuró en iteraciones cortas (sprints) con objetivos definidos: levantar requisitos mínimos viables, construir prototipos funcionales, realizar pruebas de usabilidad y ajustar funcionalidades a partir de la retroalimentación. Cada sprint concluyó con una revisión donde se consolidaron mejoras y se definieron tareas para el siguiente ciclo.

Fase 5. Comparar los resultados de la demostración con los objetivos definidos en la Fase 2. Se utilizaron métricas cuantitativas y cualitativas, las mismas que se detallan en la tabla de Técnicas e instrumentos de recolección de datos del presente capítulo.

Fase 6. En esta fase se realiza la comunicación de los resultados, los mismos que se definen en la sección Resultados del presente Trabajo.

Técnicas e instrumentos de recolección de datos

En la siguiente tabla se muestran las técnicas e instrumentos utilizados, definiendo el momento de la aplicación y el público objetivo.

Tabla 2: *Técnicas e instrumentos de recolección de datos*

Técnica	Instrumento	Objetivo	Características	Momento de aplicación	Público objetivo
Entrevista	Guía de entrevista semiestructurada	Profundizar en procesos operativos, necesidades y expectativas; obtener requisitos funcionales y no funcionales y objetivos (general y específicos) mediante entrevistas con dueños de las PYMES.	Preguntas abiertas por áreas (pedidos, inventario, delivery). Transcrita.	Fase 1: Diagnóstico (línea base).	Dueños de las PYMES.
Prueba de usabilidad	Cuestionario SUS (System Usability Scale)	Medir la usabilidad percibida (facilidad de uso, satisfacción) de la plataforma en entorno de pruebas.	10 ítems con escala Likert de 5 puntos. Versión estandarizada en español. Confiabilidad ampliamente documentada.	Fase 6: Después de completar tareas guiadas en entorno controlado.	Usuarios de prueba de la PYME piloto (administrador, personal de cocina, repartidores).

Técnica	Instrumento	Objetivo	Características	Momento de aplicación	Público objetivo
Observación estructurada	Ficha de observación de tareas guiadas	Registrar datos cuantitativos sobre desempeño en escenarios de prueba (tiempos, errores, completitud de flujos).	Diseñada para registrar: tiempos (min), conteo de errores en tareas predefinidas, tasa de éxito en flujos críticos.	Durante la ejecución de tareas guiadas en entorno controlado (staging).	Usuarios de prueba ejecutando escenarios predefinidos.
Revisión documental	Checklist de despliegue y logs del sistema	Verificar el correcto despliegue técnico en staging y analizar logs de pruebas.	Lista de verificación para servicios en la nube. Análisis de logs y métricas técnicas durante las pruebas.	Durante y después de las pruebas en entorno controlado.	Plataforma SaaS desplegada en staging y sus registros de prueba.

Consideraciones éticas

Durante el desarrollo de la investigación se respetaron principios éticos fundamentales. La participación de las PYMES fue voluntaria y se garantizó la confidencialidad de la información proporcionada. Los datos recolectados se utilizaron exclusivamente con fines académicos, asegurando el anonimato de los participantes y evitando la divulgación de información sensible.

Se aplicó consentimiento informado, explicando objetivos del estudio, naturaleza de las actividades a realizar y derechos de los participantes (retirarse en cualquier momento, solicitar correcciones o eliminación de datos). La recolección y almacenamiento de información se efectuó siguiendo prácticas de minimización de datos, con acceso restringido y protección mediante credenciales. Los

resultados se reportaron de forma agregada y sin identificadores, preservando la identidad de las PYMES. Asimismo, se contó con la autorización de los responsables de cada negocio para la aplicación de instrumentos y la evaluación del prototipo. Se evaluó el riesgo potencial de interferencia con operaciones cotidianas y se planificaron las actividades para minimizar impactos, realizando pruebas en momentos de baja carga cuando fue posible. Finalmente, se definieron periodos de retención y eliminación segura de datos conforme a buenas prácticas académicas y a la normativa aplicable.

Procedimiento

Esta sección describe el diseño del prototipo de software y su implementación, correspondiente a las Fases 3, 4 y 5 de la metodología IBD. Se profundiza en el diseño arquitectónico, el desarrollo de los componentes y la implementación en entorno real.

Para una mejor comprensión metodológica, se detallan siguiendo la estructura en 3 Etapas:

- 1) Planificación.
- 2) Desarrollo.
- 3) Validación y Despliegue.

Este flujo garantizó una transición estructurada desde la conceptualización técnica hasta la evaluación del sistema en un entorno controlado (staging).

ETAPA 1: Planificación

Diseño de la arquitectura del sistema

Stack tecnológico. Esta etapa se dedicó a la definición de los cimientos tecnológicos y estructurales de la plataforma. A partir de los requisitos funcionales y no funcionales levantados, se

realizó un análisis comparativo de tecnologías, frameworks y herramientas para cada capa del sistema. La selección final se basó en criterios de productividad del desarrollador, mantenibilidad a largo plazo, rendimiento, soporte comunitario y adecuación al contexto de las PYMES (recursos limitados, necesidad de soluciones ligeras). Las decisiones clave se resumen en la Tabla 2.

Tabla 3: *Selección tecnológica para la arquitectura de la plataforma SaaS*

Capa / Componente	Tecnología seleccionada	Alternativas evaluadas	Criterio de selección principal
Backend (API y lógica)	NestJS (Node.js/TypeScript)	Express.js (Node.js), Django (Python), Spring Boot (Java)	Arquitectura modular por defecto, tipado estático (TypeScript) para reducir errores, inyección de dependencias integrada y ecosistema robusto para APIs escalables.
Base de datos	PostgreSQL	MySQL, MongoDB (NoSQL)	Modelo relacional adecuado para la integridad de datos transaccionales (pedidos, inventario). Balance entre rendimiento, características avanzadas y costo cero (open-source).
Aplicación móvil	React Native + Expo	Flutter (Dart), desarrollo nativo (Kotlin/Swift)	Desarrollo ágil y eficiente para Android. Expo agiliza el ciclo de desarrollo (builds, testing) y el acceso a APIs nativas sin complejidad.

Capa / Componente	Tecnología seleccionada	Alternativas evaluadas	Criterio de selección principal
Panel web admin	Next.js (React) + Tailwind CSS	React (Create React App), Vue.js, Angular	Renderizado híbrido (SSR/CSR) de Next.js para mejor SEO y performance de carga; Tailwind CSS para estilizado ágil y consistente.
Autenticación	JWT (JSON Web Tokens) + bcrypt	Sesiones en servidor, OAuth 2.0 (third-party)	Stateless y escalable para APIs REST. Simplicidad de implementación y seguridad suficiente para el alcance del piloto.
Contenedorización	Docker	—	Estandarización del entorno de desarrollo y despliegue, garantizando consistencia entre equipos y facilitando el deployment en la nube.
Despliegue backend/DB	VPS (Hetzner)	AWS/Azure (cloud), Railway, Render	Costo-eficiencia y control total del servidor para un piloto, con rendimiento predecible y facturación simple.
Despliegue panel web	Vercel	Netlify, GitHub Pages	Integración nativa con Next.js, despliegue automático por Git, CDN global y SSL gratuito, minimizando tareas de DevOps.

Arquitectura de la Plataforma

Una vez establecidas las herramientas tecnológicas, se definió y estructuró una arquitectura por capas. Los componentes principales de la plataforma son:

- Aplicación móvil para clientes, repartidores y vista de cocina (aceptar o rechazar pedidos, notificar cuando esté listo para que el repartidor recoja, integrado al flujo de órdenes).
- Aplicación web principalmente para administración, con posibilidad de iniciar sesión como cliente para realizar compras desde la web.
- Servicios backends con sus respectivas APIs.

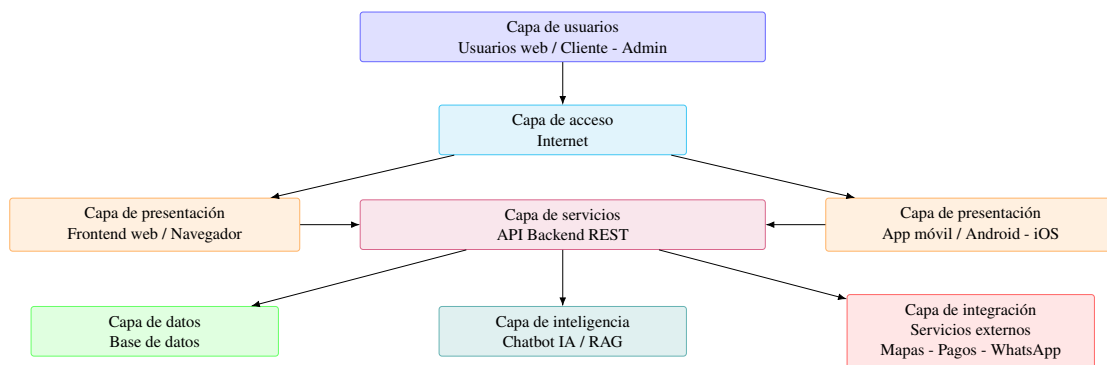


Figura 3. *Arquitectura por capas de la plataforma SaaS*

La arquitectura está organizada en cuatro capas principales: presentación (aplicación móvil y web), lógica de negocio (backend), persistencia (base de datos) e infraestructura (despliegue en nube). Cada capa es independiente y se comunica con las adyacentes mediante interfaces bien definidas.

Arquitectura de la aplicación móvil

La aplicación móvil es la interfaz principal para tres roles: cliente, repartidor y cocina. Utiliza React Native + Expo para maximizar la reutilización de código entre plataformas.

Arquitectura de la aplicación móvil

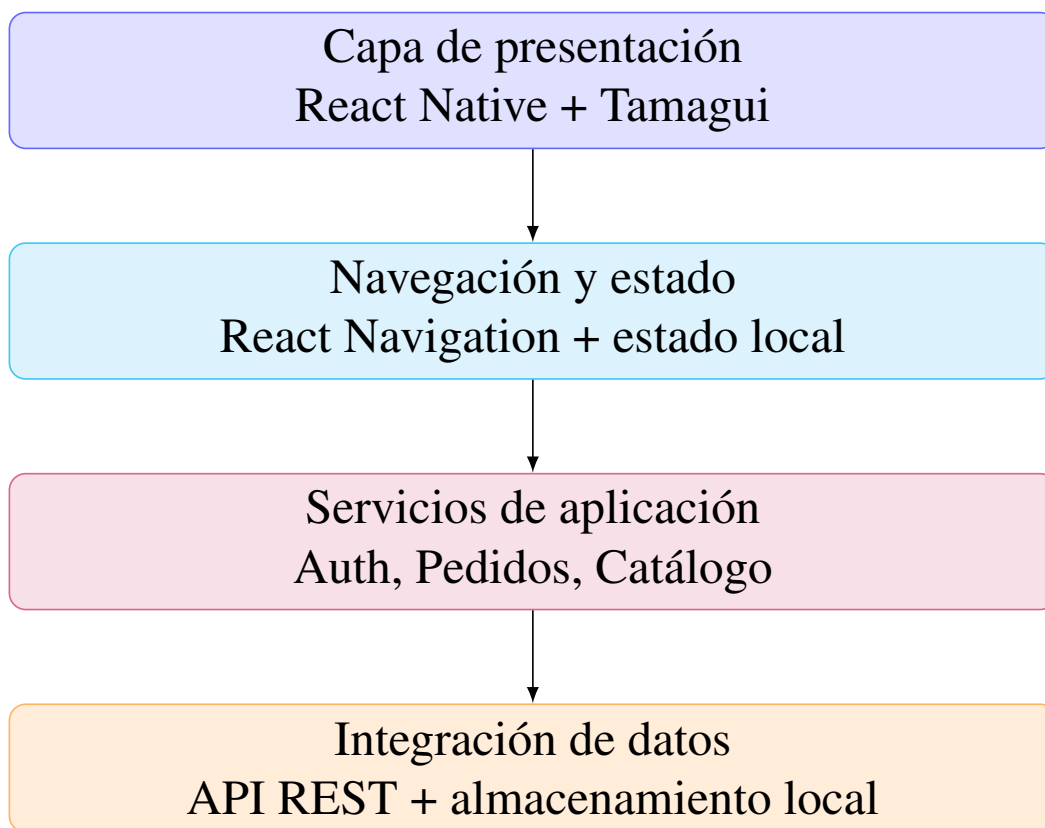


Figura 4. *Arquitectura de la aplicación móvil*

Arquitectura de la aplicación web

El panel web administrativo permite la gestión de productos, categorías, pedidos y repartidores, además de proporcionar acceso a clientes para realizar compras.

Arquitectura del panel web

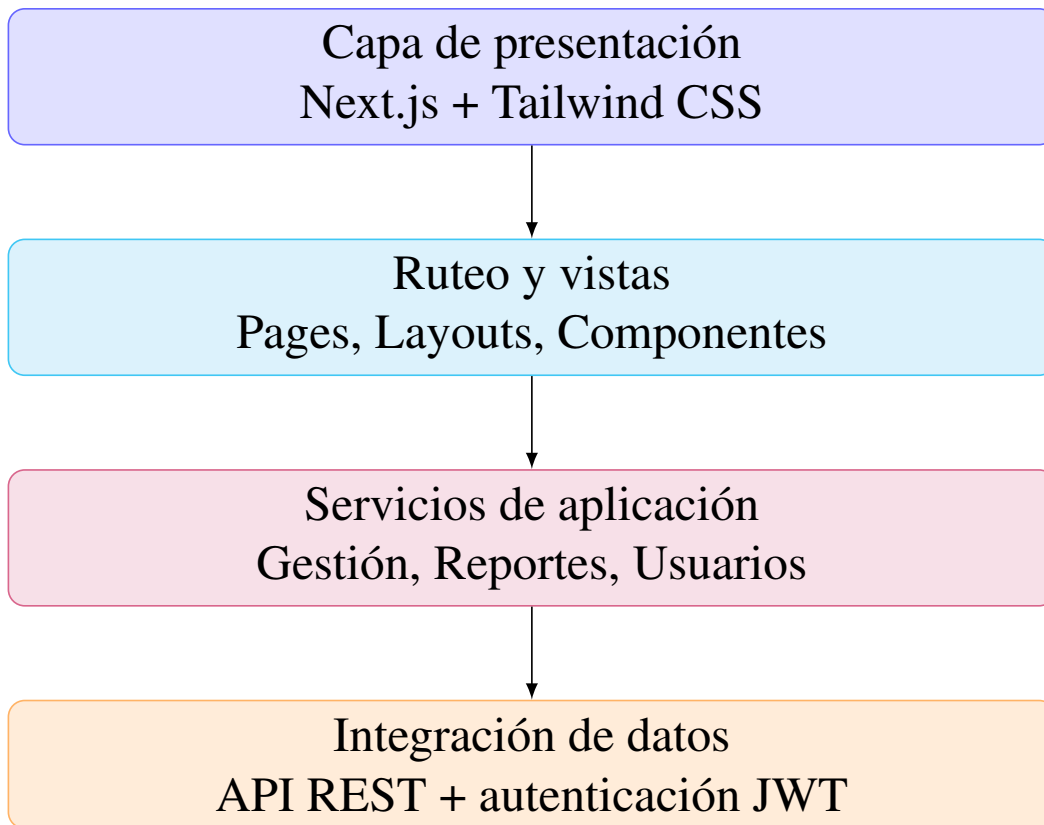


Figura 5. *Arquitectura de la aplicación web*

Arquitectura del backend

El backend expone una API RESTful que centraliza toda la lógica de negocio, autenticación y manejo de datos.

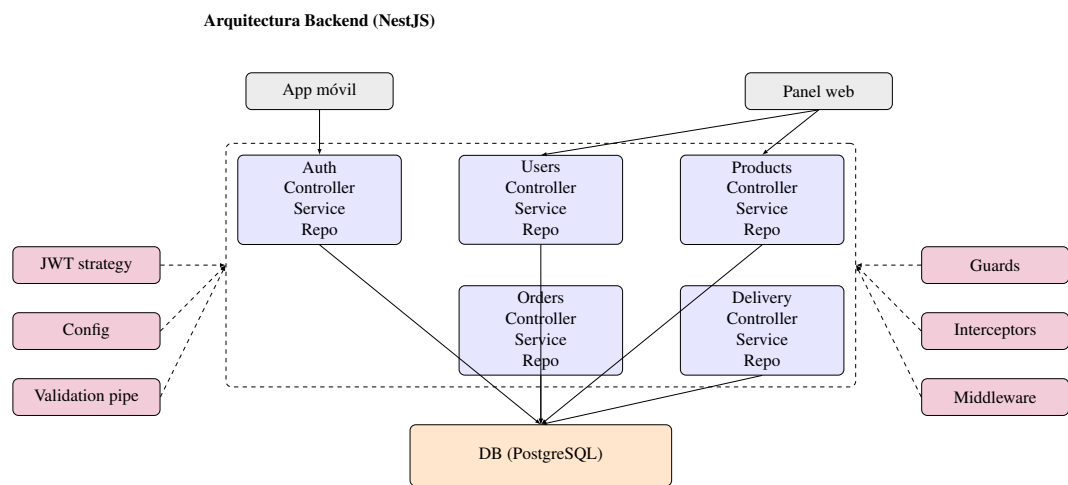


Figura 6. *Arquitectura del backend (API REST)*

Arquitectura de la base de datos

Objetivo del modelo de datos: garantizar integridad referencial, consultas eficientes y soporte para las operaciones del negocio mediante un esquema relacional estructurado.

Para esta plataforma se optó por un modelo de base de datos relacional con PostgreSQL, adecuado para representar las entidades y relaciones propias del dominio (usuarios, productos, pedidos, etc.) garantizando consistencia y permitiendo consultas complejas (p. ej., obtener todos los pedidos de un cliente filtrados por fecha, o los productos más vendidos por categoría). Se implementó utilizando Supabase como proveedor de base de datos PostgreSQL administrada, lo que facilita el despliegue y la gestión.

La decisión de usar un RDBMS sobre una base NoSQL estuvo motivada por la necesidad de integridad referencial (por ejemplo, asegurar que un pedido refiere a un usuario existente, o que un producto pertenece a una categoría válida) y la facilidad para realizar uniones (JOINS) entre tablas para extraer informes o datos combinados, algo muy útil de cara a las futuras métricas y reportes.

Modelo Entidad-Relación principal: A grandes rasgos, se definen las siguientes entidades (tablas) en el esquema de datos.

Usuario (tbl_usuarios): Representa a los usuarios de la plataforma. Incluye id (clave primaria), nombre, correo electrónico, contraseña encriptada con bcrypt, teléfono y fecha de registro.

Implementa un sistema de roles mediante relación muchos a muchos con la entidad Rol, permitiendo que un usuario tenga múltiples roles simultáneamente. Para usuarios con rol de repartidor, incluye campos específicos como imagen de perfil, descripción, tipo de vehículo y un estado de aprobación (pendiente, aprobado, rechazado) gestionado por administradores. Los usuarios se asocian a una ciudad mediante clave foránea.

Rol (tbl_roles): Define los roles del sistema (cliente, repartidor, admin, superadmin). Se relaciona con usuarios mediante tabla intermedia, soportando asignación flexible de roles. Los roles se emplean en el sistema de autorización basado en RBAC (Role-Based Access Control).

Producto (tbl_productos): Representa los productos disponibles para venta. Contiene id, nombre, descripción, imagen (URL), categoría (referencia textual), precio y stock global. Incluye un campo activo para control de disponibilidad. Se relaciona con items de pedido (OrderItem) y con stock por local (StockLocal) para control de inventario distribuido.

Categoría (tbl_categorias): Agrupa productos por tipo. Incluye id, nombre, descripción, imagen_url, ícono, color (para UI), estado activo y orden de presentación. No se vincula directamente con productos mediante clave foránea sino por referencia textual en el campo categoría del producto.

Pedido (tbl_pedidos): Registra cada orden realizada por un cliente. Contiene id, fecha del pedido, estado (pendiente, en_camino, entregado, cancelado), total, tiempo de entrega estimado, referencia al cliente (id_usuario), dirección de entrega (id_direccion) y repartidor asignado (id_repartidor, nullable). Se relaciona con items de pedido (OrderItem) que detallan los productos incluidos.

OrderItem (detalle de pedido): Tabla de relación muchos a muchos entre pedidos y productos. Almacena la cantidad de cada producto en un pedido específico, el precio unitario al momento de la compra y subtotales, manteniendo histórico de precios independiente de cambios futuros.

Ciudad (tbl_ciudades): Representa las ciudades donde opera el servicio. Contiene id y nombre. Usuarios y locales se asocian a ciudades, permitiendo segmentación geográfica de operaciones.

Local/Tienda (tbl_locales): Representa puntos de venta físicos. Incluye id, nombre, dirección, teléfono, descripción, estado activo, horarios de apertura y cierre, coordenadas geográficas (latitud/longitud) para mapas, y relación con ciudad. Se relaciona con direcciones de clientes y stock local de productos.

Dirección (tbl_direcciones): Almacena direcciones de entrega de clientes. Contiene id, usuario propietario, local asociado (opcional), calle principal, calle secundaria, número de casa, referencia, coordenadas geográficas y ciudad. Permite múltiples direcciones por usuario.

StockLocal: Gestiona inventario por local. Relaciona productos con locales y almacena la cantidad

disponible en cada punto de venta, habilitando control de stock distribuido.

Además de las relaciones, se definieron índices en campos de búsqueda frecuente (por ejemplo, estado y fecha en pedidos, correo en usuarios) para optimizar consultas, y restricciones de unicidad en claves naturales donde corresponde (emails de usuarios, nombres de roles). Se cuidó la normalización para evitar duplicidades innecesarias, manteniendo un equilibrio pragmático entre integridad y rendimiento. En entidades con alto volumen de escritura (pedidos y sus items) se contempló el uso de transacciones para asegurar consistencia en operaciones compuestas.

Implementación con TypeORM

En la implementación con TypeORM, cada entidad se refleja en una clase TypeScript con decoradores `@Entity` que mapean a las tablas correspondientes. Las relaciones se definen mediante decoradores especializados:

- `@ManyToOne` y `@OneToMany` para relaciones uno a muchos (ej: Usuario-Pedidos, Pedido-OrderItems)
- `@ManyToMany` con `@JoinTable` para relaciones muchos a muchos (ej: Usuario-Roles)
- `@JoinColumn` para especificar claves foráneas explícitas

Por ejemplo, la entidad Order (Pedido) define `@ManyToOne(() => User, user => user.orders)` para referenciar al cliente, con un `@JoinColumn` indicando `id_usuario` como clave foránea. De forma similar, referencia al repartidor asignado mediante otra relación a User (nullable), distinguiendo roles mediante el sistema RBAC implementado.

La configuración de TypeORM permite sincronización automática del esquema en desarrollo (`synchronize: true` cuando `NODE_ENV !== 'production'`), facilitando iteraciones rápidas. Las entidades emplean transformadores personalizados para tipos específicos (p. ej., convertir strings decimales de PostgreSQL a números JavaScript en campos de precio).

Integridad referencial y reglas de negocio

Se definieron claves foráneas para garantizar consistencia: un pedido siempre referencia un usuario y dirección válidos. Para mantener integridad histórica, no se permite eliminación física de usuarios con pedidos; en su lugar se emplea un campo activo o estado para desactivación lógica. Las relaciones críticas previenen borrados cascada no deseados, protegiendo el histórico transaccional.

La arquitectura soporta operaciones distribuidas por ciudad y local. Aunque actualmente opera para un cliente piloto, la estructura con entidades Ciudad y Local sienta las bases para expansión geográfica. El control de stock por local (StockLocal) permite gestión descentralizada de inventario, relevante para cadenas o franquicias futuras.

Estrategia de escalabilidad

Si bien la implementación actual emplea una única base de datos para el cliente piloto, la arquitectura contempla escalabilidad futura mediante dos enfoques posibles:

- 1) **Base de datos por cliente:** Estrategia de multi-tenancy mediante bases de datos separadas por cliente empresarial. NestJS/TypeORM soportan conexiones dinámicas, permitiendo que cada empresa opere en su propia instancia PostgreSQL, garantizando aislamiento total de datos y cumplimiento de normativas de privacidad.
- 2) **Segmentación geográfica:** Escalar horizontalmente mediante réplicas por región geográfica, aprovechando la entidad Ciudad para enrutamiento de consultas.

La configuración actual con Supabase facilita esta migración, al soportar múltiples proyectos de base de datos administrados de forma independiente.

Finalmente, se contemplaron estrategias de respaldo automático (backups diarios mediante Supabase), recuperación ante desastres y mecanismos de migración de esquema controlada mediante archivos SQL versionados (ver carpeta migrations/ con ejemplos como create-stock-local-table.sql), abandonando sincronización automática en producción.

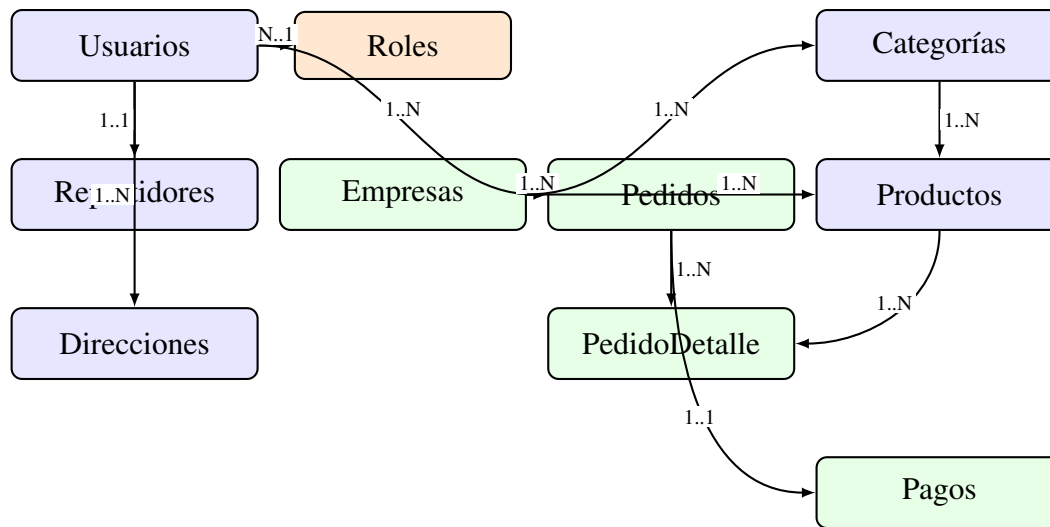


Figura 7. Diagrama Entidad-Relación (ER) de la base de datos

Roles, permisos y seguridad (JWT, RBAC)

La plataforma implementa un esquema de roles y permisos (RBAC) para garantizar que cada usuario solo realice las acciones que le corresponden (Logto, s.f.). RBAC gestiona "quién puede hacer qué" agrupando permisos en roles; a cada usuario se le asigna uno o varios roles y cada rol confiere permisos sobre funciones, APIs o datos.

Administrador (ADMIN): Usuario del panel web con privilegios completos. Puede crear y editar productos y categorías, ver todos los pedidos, gestionar repartidores (aprobar/rechazar) y configurar la plataforma. Existe un rol adicional de Superadmin con acceso irrestricto al sistema completo.

Cliente (CLIENTE): Usuario de la app móvil que realiza pedidos. Puede registrarse o iniciar sesión, ver y buscar productos, crear pedidos, consultar su estado y editar su perfil.

Repartidor (DELIVERY): Usuario de la app móvil que toma y entrega pedidos. Puede ver pedidos pendientes, aceptar/declinar encargos, marcar entregas y revisar su historial. Requiere aprobación administrativa antes de operar.

Cocina (COCINA): Usuario de la app móvil (vista de cocina) encargado de la gestión de pedidos. Recibe las órdenes nuevas y tiene la capacidad de aceptarlas o rechazarlas; una vez preparadas, marca el pedido como listo para que el repartidor proceda a recogerlo y entregarlo al cliente,

actualizando el estado del pedido en el flujo del sistema.

Estos roles se implementan mediante una tabla independiente (tbl_roles) y una relación muchos a muchos con usuarios, permitiendo que un mismo usuario posea múltiples roles simultáneamente. La tabla de roles almacena los identificadores estándar (cliente, repartidor, admin, superadmin) con restricción de unicidad. Esta arquitectura flexible facilita la gestión de permisos compuestos; por ejemplo, un usuario puede ser simultáneamente cliente y administrador.

La autorización por roles se implementa de forma declarativa en endpoints críticos, con mensajes de error consistentes y registro de intentos no autorizados para auditoría. En cuanto a autenticación, se definieron políticas de expiración de tokens equilibradas entre seguridad y usabilidad; por simplicidad, inicialmente no se emplean tokens de actualización (refresh), aunque se prevé su inclusión si el uso lo amerita. Se recomienda almacenar credenciales y tokens de forma segura en cliente (cookies HttpOnly en web, almacenamiento seguro en móvil) y evitar exposición en logs.

Para la autenticación y autorización, utilizamos JWT (JSON Web Tokens) como mecanismo de autenticación stateless. JWT es un estándar abierto (RFC 7519) ampliamente utilizado para transmitir información de forma segura en forma de token JSON firmado. De hecho, la autenticación con tokens JWT se ha convertido en un estándar de facto para proteger los endpoints sensibles de aplicaciones web y APIs (Aguilera, .xm XX implementamos JWT mediante la integración de Passport.js en NestJS con la estrategia passport-jwt.

ETAPA 2: Desarrollo

Desarrollo de la Plataforma SaaS

Los Objetivos Específicos 3, 4 y 5 se centraron en la construcción iterativa e integración de todos los componentes del sistema. Se siguió una metodología ágil, organizando el trabajo en sprints de dos semanas (Anexo 5). El desarrollo fue modular y paralelo. Configuración inicial: se establecieron los tres repositorios (backend, móvil, web), se configuraron las herramientas de

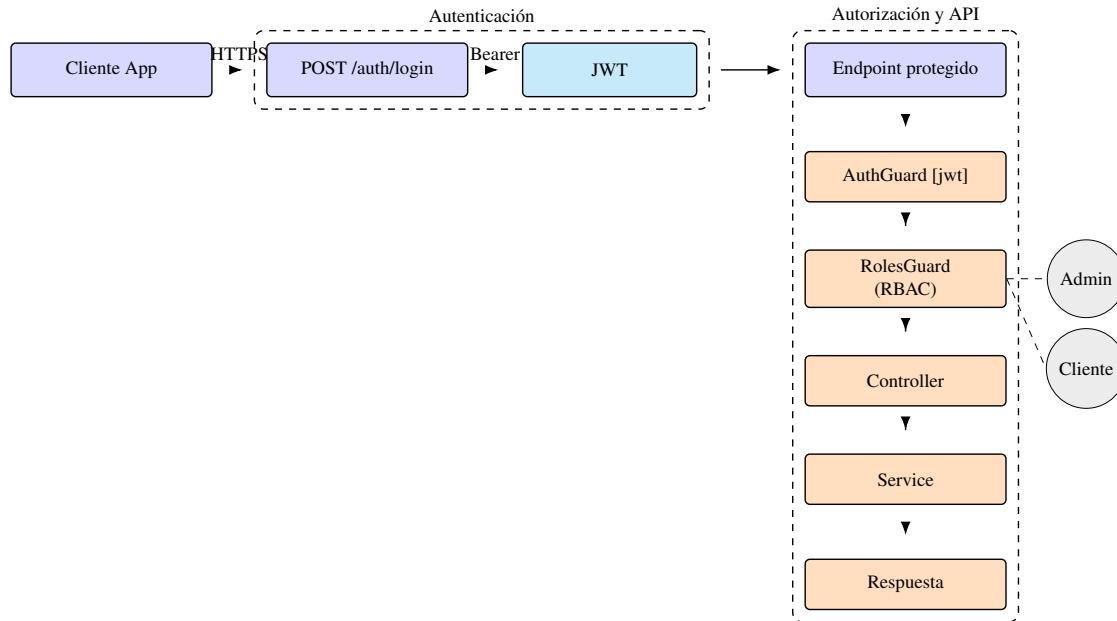


Figura 8. *Flujo de autenticación y autorización (JWT + RBAC)*

contenedorización (Docker) y se definió la estructura base de la base de datos en PostgreSQL.

Desarrollo del frontend móvil (React Native + Expo)

Objetivo del frontend: ofrecer una experiencia clara y eficiente para clientes, repartidores y personal de cocina, con navegación predecible, validaciones tempranas y comunicación robusta con la API. El frontend móvil se implementó con React Native y Expo, lo que permite desarrollar una aplicación nativa multiplataforma a partir de una sola base de código.

React Native facilita la reutilización entre iOS y Android sin comprometer el rendimiento; Expo simplifica el ciclo de vida de la aplicación al ofrecer herramientas integradas para compilar, desplegar y probar sin configurar proyectos nativos desde cero (campusMVP, s.f.). Además, Expo expone funcionalidades nativas desde JavaScript (cámara, notificaciones, sensores) y habilita un flujo ágil con recarga en vivo y publicación mediante enlaces o códigos QR, favoreciendo la colaboración y las pruebas (campusMVP, s.f.).

La aplicación ofrece flujos diferenciados para tres perfiles: Cliente, Repartidor y Cocina, además de pantallas de inicio de sesión y registro. Se incorporó un módulo o vista de *cocina* en la aplicación

móvil que permite aceptar o rechazar pedidos y notificar cuando el pedido está listo para que el repartidor recoja, conectado al flujo de solicitud de órdenes. Así, cuando un cliente realiza un pedido, la cocina puede visualizarlo, aceptarlo o rechazarlo, y al marcarlo como listo el repartidor puede ir a recogerlo para la entrega. La navegación se gestiona con la librería de React Native (p. ej., React Navigation), que permite transicionar entre vistas, mantener historiales y pasar parámetros. Cada rol visualiza únicamente las opciones pertinentes; esta segregación mejora la experiencia y refuerza la seguridad, complementada con validaciones en el backend.

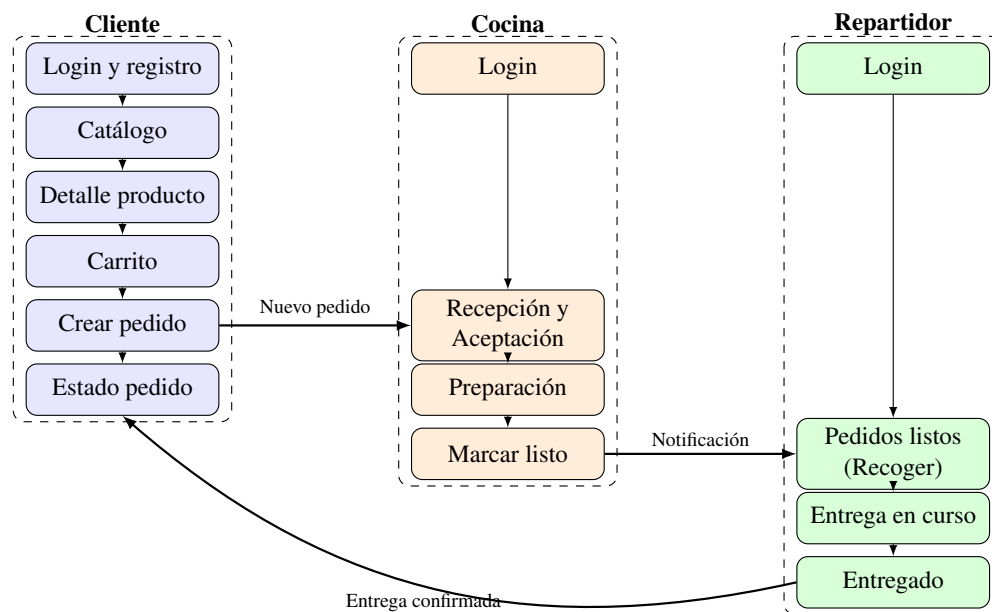


Figura 9. Flujo de la interfaz móvil por rol (Cliente, Repartidor, Cocina)

La arquitectura del frontend móvil privilegia la simplicidad y la previsibilidad del estado. Se emplea estado local y patrones de elevación de estado donde corresponde, con manejo de formularios controlados y validación básica en cliente (tipos de entrada, obligatoriedad, formatos). La comunicación con la API se realiza mediante solicitudes HTTP que envían y reciben JSON; se contemplan casos de error (credenciales inválidas, falta de conectividad, respuestas 4xx/5xx) con mensajes claros al usuario y estrategias de reintento cuando es pertinente. La interfaz se diseñó con principios de consistencia visual y accesibilidad, cuidando contrastes, tamaños de toque y retroalimentación tras acciones. Para el diseño visual se empleó Tamagui, biblioteca de UI y

sistema de estilos compatible con React Native, que provee componentes y temas para construir interfaces coherentes y responsivas (Tamagui, s.f.). En el proyecto se estilizaron botones, tarjetas y formularios con componentes Tamagui, logrando consistencia y mantenibilidad. Su sistema de temas facilita la personalización visual (colores, tipografía) y asegura una apariencia uniforme en la aplicación Android, adaptándose a las necesidades de identidad del negocio.

Desarrollo del frontend panel web (Next.js + Tailwind CSS)

Objetivo del panel web: proveer principalmente herramientas de administración para gestionar catálogo, pedidos y repartidores, con formularios validados, vistas responsivas y flujos CRUD claros; además, se permite iniciar sesión como cliente para realizar compras desde la web.

Se desarrolló un sitio web con React y Next.js cuya función principal es la administración del negocio (panel administrativo). También se implementó la posibilidad de que los clientes inicien sesión desde la web para realizar pedidos desde el navegador, complementando la opción de la aplicación móvil. Se aprovechó el renderizado del lado del servidor (SSR) en vistas específicas de dashboard cuando resultó conveniente.

El estilado se realizó con Tailwind CSS, framework utility-first que permite componer diseños rápidos y consistentes mediante clases utilitarias (Tailwind CSS, s.f.). Su configuración admite alta personalización, útil para adaptar paletas o temas por cliente.

En el panel se aplicaron formularios validados con reglas de negocio simples (campos requeridos, formatos, rangos) y se implementaron flujos habituales de CRUD con confirmaciones y mensajes de éxito/error.

La estructura de navegación organiza secciones por dominio (productos, categorías, pedidos, repartidores) y proporciona filtros básicos (por estado, fecha) para facilitar la gestión. Se cuidó la responsividad para operar en distintos tamaños de pantalla, y se diseñaron tablas y listas con paginación prevista para escenarios de datos voluminosos.

El panel permite crear y editar productos, gestionar categorías y aprobar o rechazar repartidores que solicitan unirse. Los formularios incorporan validaciones de campos; las vistas de lista presentan pedidos en curso y solicitudes pendientes.

Cuando un repartidor se registra desde la app móvil, su cuenta queda "pendiente" hasta la revisión administrativa, tras la cual puede ser habilitada o rechazada, garantizando control de calidad.

Ambos frontends consumen el mismo backend mediante API REST y JSON. La separación de responsabilidades mantiene el frontend enfocado en la experiencia y presentación, mientras la lógica de negocio y la persistencia residen en el backend.

En materia de rendimiento y experiencia, se optimizaron listas y vistas frecuentes y se cuidó la retroalimentación visual en operaciones que pueden tardar (carga de catálogo, envío de pedidos). Se delinearon buenas prácticas para el manejo de imágenes (compresión, tamaños adecuados) y para el control de estados vacíos y errores de red, mejorando la robustez de la interfaz en condiciones reales.

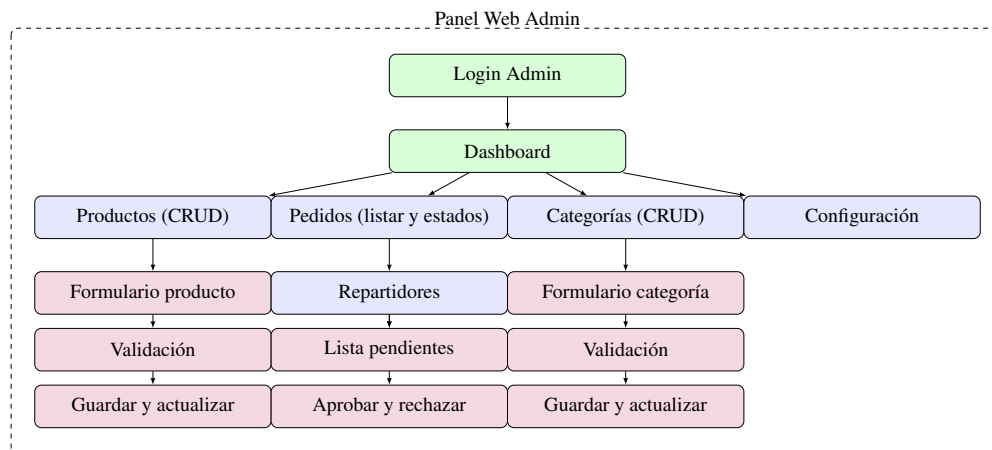


Figura 10. Flujo del panel web administrativo

Desarrollo del backend (NestJS, capas, módulos)

Objetivo del backend: exponer servicios estables y seguros, aplicar reglas de negocio y coordinar persistencia con una arquitectura modular mantenible.

El backend se implementó con NestJS, framework de Node.js de arquitectura modular que promueve buenas prácticas mediante el uso de TypeScript, decoradores e inyección de dependencias. Frente a Express puro, NestJS aporta una estructura clara basada en módulos, controladores y servicios, facilitando la escalabilidad y el mantenimiento del código por dominios de negocio (Usuarios, Productos, Pedidos, Autenticación, entre otros).

Cada módulo agrupa tres componentes fundamentales: un controlador que define endpoints y atiende solicitudes HTTP; un servicio que concentra la lógica de negocio; y una capa de datos o repositorio que interactúa con la base de datos, ya sea mediante un ORM o consultas específicas (Dragos, s.f.). Siguiendo este patrón, el módulo de Autenticación expone rutas para login y registro en su controlador; el servicio valida credenciales y emite tokens JWT; y se apoya en el módulo de Usuarios para verificar identidad. De modo análogo, el módulo de Productos ofrece rutas para listar y crear productos (acción reservada al administrador), mientras su servicio aplica reglas de negocio y sus entidades mapean a tablas mediante el ORM.

La capa de control incluye manejo de errores y respuestas consistentes; se emplean filtros de excepciones para capturar y formatear fallos (validación, autorización, acceso a datos) en códigos HTTP adecuados. Los servicios encapsulan reglas de negocio con claridad, evitando dependencias con detalles de transporte HTTP. La configuración de la aplicación se centraliza con lectura de variables de entorno, diferenciando perfiles de ejecución (desarrollo, pruebas, producción) y resguardando credenciales. El uso integral de TypeScript habilita detección temprana de errores y mejor navegación del código.

Se definieron DTOs para tipar y validar datos de entrada y salida; la integración con classvalidator y class-transformer, junto con pipes de validación global, permite rechazar automáticamente solicitudes mal formadas y entregar al controlador datos ya validados. La inyección de dependencias mantiene el código desacoplado y testeable. Marcando clases como @Injectable(), estas pueden solicitarse en constructores de otras clases y Nest provee instancias adecuadas. Por ejemplo, OrdersService inyecta UsersRepository, AddressRepository y ProductsRepository

mediante decorador `@InjectRepository`, permitiendo operaciones de consulta y persistencia sin acoplamiento directo a la implementación del ORM.

Las responsabilidades se mantienen unidireccionales: el controlador recibe la petición HTTP, valida tokens JWT mediante guards, aplica decoradores de roles (`@Roles`) para autorización y delega al servicio; el servicio ejecuta reglas de negocio (validar stock, calcular totales, asignar repartidores) y coordina repositorios; la capa de datos interactúa exclusivamente con PostgreSQL mediante TypeORM (Dragos, s.f.; Q2B Studio, s.f.). Esta separación, alineada con Clean Architecture, evita mezclas de responsabilidades y facilita el reemplazo de componentes sin afectar capas superiores, favoreciendo mantenibilidad y escalabilidad.

Asimismo, simplifica las pruebas unitarias al permitir testear servicios o controladores en aislamiento mediante simulaciones. La persistencia se maneja con TypeORM, integrado mediante `@nestjs/typeorm`. Las entidades TypeScript decoradas representan tablas; los repositorios y el QueryBuilder facilitan operaciones sin escribir SQL manual. TypeORM soporta múltiples motores (PostgreSQL, MySQL, etc.) y ofrece migraciones de esquema; aunque inicialmente se usó sincronización automática, se prevé incorporar migraciones 23 controladas.

La organización por módulos comprende:

- **AuthModule:** Gestiona registro, login y emisión de tokens JWT. Emplea `bcrypt` para hash de contraseñas (salt rounds: 10) y `JwtModule` configurado con clave secreta y expiración. Implementa estrategia `Passport-JWT` para validar tokens en headers `Authorization`.
- **UsersModule:** Administra usuarios y relación con roles. Provee endpoints protegidos (solo admin/superadmin) para CRUD de usuarios, gestión de roles y consulta de perfiles. Integrado estrechamente con `AuthModule`.
- **ProductsModule:** Gestión de catálogo de productos. Endpoints públicos para listar y consultar productos; endpoints protegidos (admin/superadmin) para crear, actualizar y eliminar. Integra `StockLocal` para control de inventario por local.

- **CategoriesModule:** CRUD de categorías de productos. Endpoints protegidos por roles, con listado público para clientes.
- **OrdersModule:** Creación, consulta y gestión de estados de pedidos. Clientes crean pedidos; repartidores consultan asignaciones; admins visualizan todos los pedidos con filtros por estado. Maneja relaciones complejas con OrderItems, Users, Addresses y Products.
- **AddressesModule:** Gestión de direcciones de entrega por usuario. Cada usuario maneja sus propias direcciones; validación de pertenencia mediante guards.
- **CitiesModule:** CRUD de ciudades. Listado público; creación/eliminación restringida a admin/superadmin.
- **StoresModule:** Gestión de locales comerciales con información geográfica (coordenadas, horarios). Usado para asignación de pedidos y control de stock distribuido.
- **AdminModule:** Operaciones administrativas específicas como aprobación/rechazo de repartidores, reportes y configuraciones de negocio.

NestJS se integra con Passport.js para la autenticación; se definió una estrategia JWT personalizada (JwtStrategy) que extrae y valida el token, decodifica el payload y adjunta el usuario al objeto request para uso en controladores y guards.

Además, se emplearon Guards, Interceptors y Middleware en puntos específicos:

- **JwtAuthGuard:** Extiende AuthGuard('jwt') de Passport. Protege endpoints requiriendo token válido en header. Usado en mayoría de rutas privadas.
- **RolesGuard:** Guard personalizado que verifica roles del usuario autenticado contra los requeridos por decorador @Roles. Consulta roles desde el payload JWT y valida permisos antes de permitir acceso al endpoint. Implementa lógica para superadmin (acceso total) y validación condicional por ciudad para admins.

- **ThrottlerModule:** Configurado globalmente para rate limiting (100 requests por minuto por IP), mitigando abuso y ataques de fuerza bruta.
- **ValidationPipe global:** Activado en bootstrap de la aplicación para validar y transformar DTOs automáticamente usando class-validator y class-transformer. Rechaza requests con datos inválidos retornando errores HTTP 400 descriptivos.

Se contempla incorporar interceptores para logging de auditoría, medición de tiempos de respuesta y formateo consistente de respuestas JSON.

En validación de datos, se activó ValidationPipe global con opciones whitelist (elimina propiedades no declaradas en DTOs) y forbidNonWhitelisted (rechaza requests con campos extra), robusteciendo la API ante entradas malformadas y ataques de inyección de datos.

Configuración de seguridad y despliegue

Se implementaron medidas de endurecimiento para producción:

- **CORS:** Configurado por origen permitido, restringiendo acceso desde dominios no autorizados.
- **Rate Limiting:** ThrottlerModule con límite de 100 requests/minuto por IP.
- **Conexión SSL a base de datos:** Configuración de TypeORM con ssl: true y rejectUnauthorized: false para Supabase, con opciones de pool (max: 20 clientes, timeout: 30s).
- **Variables de entorno:** ConfigModule global carga credenciales desde archivo .env (DB_HOST, DB_PORT, DB_USERNAME, DB_PASSWORD, DB_NAME, JWT_SECRET, NODE_ENV), evitando hard-coding de secretos.
- **Sincronización condicional:** TypeORM synchronize activo solo en desarrollo; en producción se emplean migraciones SQL versionadas.

El backend se despliega como aplicación Node.js independiente. Se contempló Docker para contenerización (facilitando despliegue reproducible en VPS), con imágenes multi-stage para optimizar tamaño. La configuración distingue perfiles de ejecución mediante `NODE_ENV` (development, production), ajustando logging y optimizaciones según entorno.

Para producción se planificó despliegue en VPS (Hetzner) con proceso administrado por PM2 o similar, proxy inverso Nginx para SSL/TLS y balanceo, y monitoreo de salud del servicio. La base de datos PostgreSQL administrada por Supabase elimina complejidad de gestión de infraestructura de datos.

Ejemplo de flujo completo: Creación de pedido

Para ilustrar la arquitectura en acción, se describe el flujo de creación de un pedido:

- 1) Cliente autenticado envía POST a `/api/orders` con datos del pedido (productos, cantidades, dirección) en JSON.
- 2) `JwtAuthGuard` valida token en header Authorization, extrae `userId` y roles, adjunta al request.
- 3) `ValidationPipe` transforma y valida `CreateOrderDto`, rechazando datos inválidos.
- 4) `OrdersController` delega a `OrdersService.create()`, pasando `userId` del contexto.
- 5) `OrdersService` verifica: usuario existe, dirección pertenece al usuario, productos existen con stock suficiente.
- 6) Se crea entidad `Order` con estado 'pendiente', calculando total desde precios actuales de productos.
- 7) Se crean entidades `OrderItem` por cada producto, vinculando al pedido.
- 8) `TypeORM` persiste en transacción (`Order` + `OrderItems`), garantizando atomicidad.
- 9) Se retorna respuesta 201 Created con detalles del pedido creado.

Este flujo ejemplifica separación de responsabilidades, validación automática, uso de relaciones ORM y lógica de negocio encapsulada en servicios.

Finalmente, se prevé definir estrategias de respaldo y recuperación ante desastres acordes al tamaño del despliegue, así como mecanismos de migración segura para cambios de esquema futuros (migraciones versionadas, ventanas de mantenimiento y validación posterior a la actualización).

Integración de módulos funcionales: Una vez establecida la comunicación básica entre frontend y backend vía API REST, se implementaron las funcionalidades específicas:

Módulo de Pedidos:

El flujo general del proceso de pedido contempla la creación, estados (pendiente, en camino, entregado) y asociación con geolocalización.

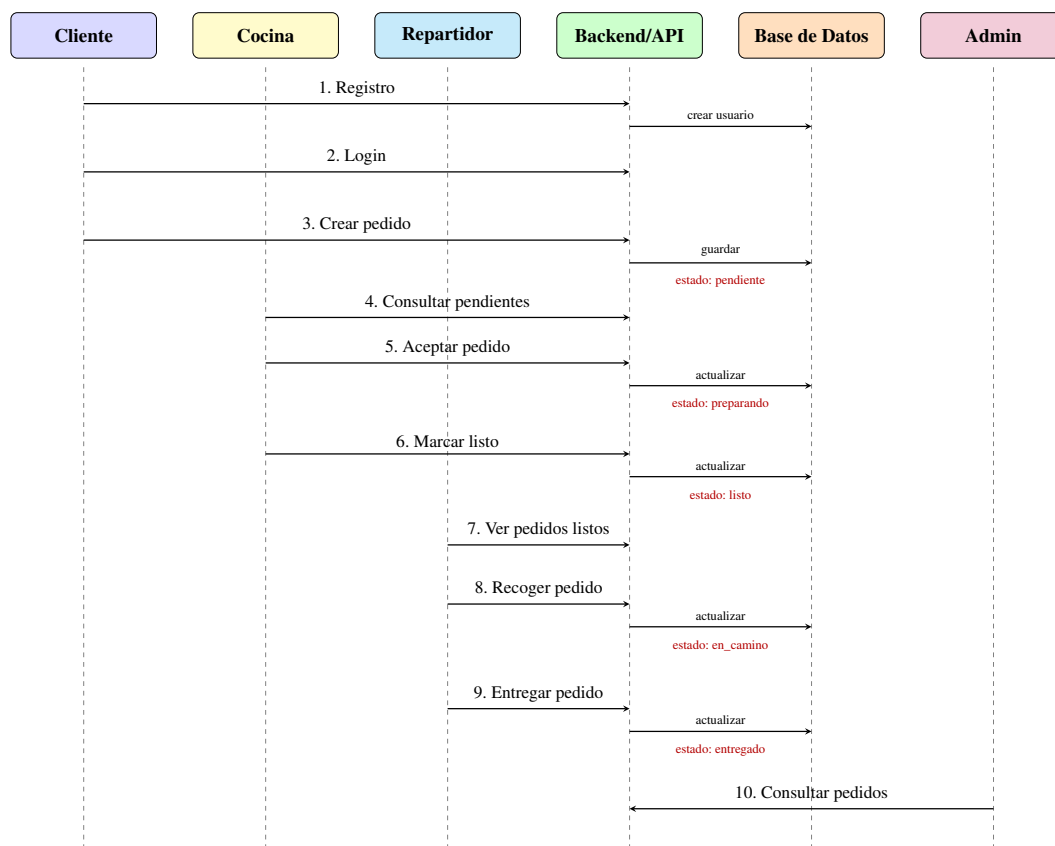


Figura 11. *Diagrama de secuencia: flujo completo del proceso de pedido*

Módulo de Inventario:

Lógica para descontar stock y prevenir ventas de productos agotados.

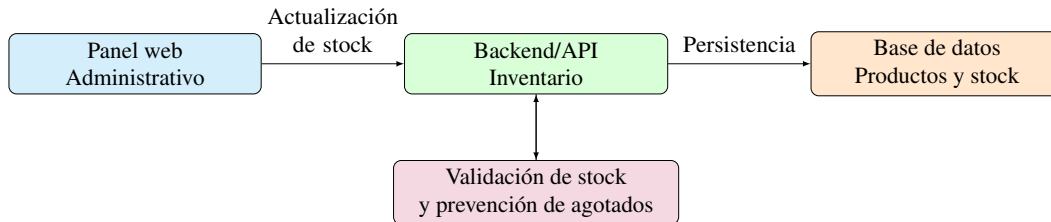


Figura 12. *Lógica del módulo de inventarios y control de stock*

Módulo de Repartidores:

Proceso de registro, aprobación administrativa y asignación manual de pedidos.

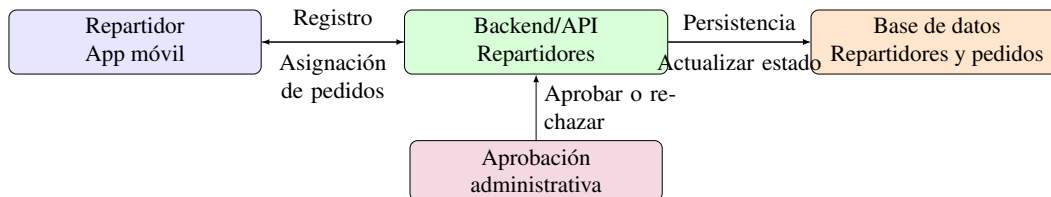


Figura 13. *Flujo del módulo de repartidores*

Dashboard Analítico:

Consultas SQL y visualización de KPIs (total de pedidos, ventas por día).

Chatbot RAG por WhatsApp:

Desarrollo de un servicio independiente utilizando LangChain, integración de un modelo de lenguaje (LLM) y conexión con la base de conocimiento de la plataforma, desplegado como canal de atención a través de WhatsApp (WhatsApp Business Cloud).

Pruebas técnicas: Se realizaron pruebas unitarias en servicios críticos y pruebas de integración manuales para garantizar la correcta comunicación entre componentes.

ETAPA 3: Validación y Despliegue

Arquitectura del despliegue

Se configuró un VPS en Hetzner con Docker para alojar el backend y la base de datos PostgreSQL. El panel web se desplegó automáticamente en Vercel. La aplicación móvil se compiló y distribuyó como archivo APK para su instalación en dispositivos Android del local piloto.

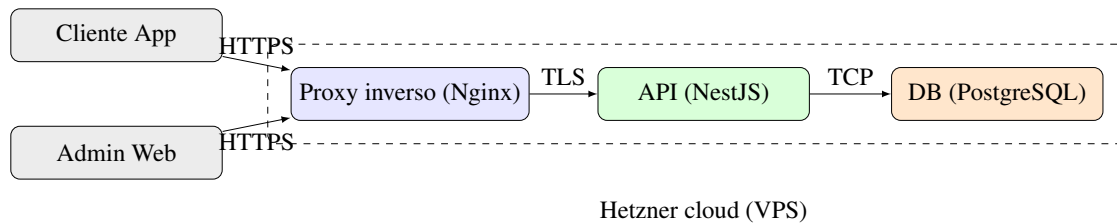


Figura 14. *Arquitectura de despliegue (VPS, Vercel, APK)*

Se recomienda un proxy inverso para gestionar certificados TLS y enrutar tráfico, así como automatizar despliegues con pipelines que ejecuten verificación y pruebas mínimas antes de publicar cambios. En resumen, el backend NestJS proporcionó una arquitectura modular robusta. Cada nueva funcionalidad se implementa añadiendo o modificando un módulo sin afectar otros, lo que agiliza el desarrollo colaborativo y reduce errores colaterales. La combinación de TypeScript, inyección de dependencias y la estructura en capas (Controlador-Servicio-Repositorio) hace que el código sea más fácil de seguir y depurar. NestJS, con su enfoque de convención sobre configuración, mantuvo aspectos como la seguridad, validación y estructura de proyecto cubiertos por diseño.

Despliegue en staging y validación técnica

Esta etapa tuvo como propósito evaluar el sistema integral en entorno controlado (staging) mediante tareas guiadas y escenarios de prueba predefinidos. El procedimiento completo (despliegue, orientación, escenarios, tareas guiadas, recolección de datos y monitoreo técnico) se documenta en el Apéndice J. Para mantener este capítulo conciso, se resume el proceso en tres acciones clave:

- **Despliegue controlado:** Backend y base de datos en VPS con Docker; panel web en Vercel; app móvil en APK para pruebas.
- **Ejecución de escenarios:** Usuarios de prueba ejecutaron flujos críticos (pedido, cocina, reparto, administración y chatbot).
- **Registro de métricas:** Se capturaron tiempos, errores, completitud de tareas y logs técnicos para análisis posterior.

El detalle de las tareas evaluadas y sus criterios de éxito se presenta en el Apéndice J (Tabla 7).

Resultados

Introducción

El presente capítulo expone los resultados obtenidos de la investigación en torno a la digitalización de las PYMES gastronómicas de la ciudad de Portoviejo y al impacto de la plataforma SaaS desarrollada. Se estructura en tres apartados principales: (1) resultados del diagnóstico de digitalización previo a la implementación; (2) resultados de la validación de usabilidad y adopción; y (3) resultados técnicos del software, incluyendo pruebas, métricas de disponibilidad y rendimiento. La exposición es objetiva y descriptiva; las interpretaciones y relaciones con la literatura se abordan en el capítulo de Discusión.

La plataforma validada está constituida por tres componentes integrados: (1) un **backend** (API REST con NestJS y PostgreSQL) que centraliza autenticación por roles (administrador, cocina, repartidor, cliente), gestión de productos, categorías, pedidos, direcciones, locales y repartidores; (2) un **panel web** (Next.js) que permite a los administradores gestionar productos, categorías, pedidos, repartidores y reportes, y a los clientes realizar pedidos desde la tienda en línea (carrito, checkout con pago en efectivo o transferencia, historial de pedidos y direcciones); (3) una **aplicación móvil** (Expo/React Native) con vista específica para cocina (listar pedidos por ciudad, confirmar y preparar, marcar listo para recoger), flujo para repartidores (pedidos disponibles y asignados, aceptar pedido, marcar recogido, en camino y entregado, estado activo/inactivo) y flujo para clientes (productos por local, carrito, checkout, seguimiento del pedido). Los tres componentes consumen la misma API y fueron utilizados de forma conjunta por la PYME piloto durante el período de validación.

Resultados del diagnóstico de digitalización

En esta sección se presentan los resultados del análisis del estado de digitalización de la PYME gastronómica piloto antes de la implementación de la plataforma, identificando el nivel de adopción tecnológica, procesos operativos vigentes y brechas detectadas.

Caracterización de la PYME piloto

La PYME piloto seleccionada corresponde a un establecimiento gastronómico formalmente constituido en la ciudad de Portoviejo, con 3 años de operación, servicio de consumo en local y delivery. El negocio opera con 1 local físico, empleando a 8 personas: 1 administrador/propietario, 3 personas en cocina, 2 en atención al cliente y 2 repartidores. Atiende un promedio de 45-60 pedidos diarios, de los cuales aproximadamente el 65 % son delivery y 35 % consumo en local.

Estado de digitalización previo

La entrevista semiestructurada con el responsable del negocio y la observación de procesos operativos revelaron el siguiente estado de digitalización:

Gestión de pedidos:

Los pedidos se recibían mediante tres canales fragmentados: (1) llamadas telefónicas (40 % de pedidos), (2) mensajes de WhatsApp personal del propietario (45 %), y (3) pedidos presenciales en local (15 %). No existía un sistema centralizado de registro; los pedidos se anotaban en cuadernos físicos, generando un tiempo promedio de 5-7 minutos por pedido y una tasa de errores estimada del 18 % (información incompleta, direcciones incorrectas, malentendidos en especificaciones).

Comunicación con cocina:

La transmisión de pedidos desde el punto de recepción hacia cocina se realizaba verbalmente o mediante notas manuscritas. El personal de cocina reportó no contar con visibilidad clara del estado de pedidos pendientes ni capacidad de rechazo cuando la demanda superaba la capacidad operativa, generando retrasos promedio de 12-15 minutos en períodos de alta demanda.

Gestión de repartidores:

La asignación de pedidos a repartidores se realizaba manualmente por el administrador mediante llamadas o mensajes de WhatsApp. No existía control automatizado de disponibilidad de repartidores ni registro digital del estado de entregas. El tiempo promedio de entrega reportado era de 40-50 minutos, sin visibilidad en tiempo real para clientes.

Control de inventarios:

El inventario se gestionaba mediante registros en hojas de cálculo actualizadas manualmente al final del día, sin integración con el sistema de pedidos. Esto resultaba en disponibilidad de productos desactualizada y ventas ocasionales de productos agotados (estimado 8-12 % de los pedidos requerían modificaciones por falta de stock).

Atención al cliente:

No existía un canal automatizado de atención. Las consultas sobre estado de pedidos, horarios y menú se respondían manualmente por el propietario vía WhatsApp o llamadas, generando carga operativa y tiempos de respuesta variables (5-30 minutos según disponibilidad).

Herramientas digitales en uso:

El negocio utilizaba únicamente WhatsApp Business (sin automatización), hojas de cálculo en Google Sheets para inventarios y contabilidad básica, y no contaba con sitio web propio ni

aplicación móvil. No se empleaban plataformas de gestión integral ni sistemas de punto de venta digitales.

Necesidades operativas identificadas

A partir del diagnóstico, se identificaron las siguientes necesidades operativas prioritarias que guiaron el diseño de la plataforma:

1. **Centralización de pedidos:** Unificar los tres canales de recepción (teléfono, WhatsApp, presencial) en una plataforma única accesible desde web y móvil.
2. **Reducción de errores y tiempos:** Disminuir el tiempo de registro de pedidos de 5-7 min a menos de 3 min y reducir la tasa de errores del 18 % a menos del 10 %.
3. **Vista operativa de cocina:** Proveer a cocina de una interfaz digital para visualizar pedidos pendientes, aceptar/rechazar según capacidad y marcar pedidos listos para recogida.
4. **Optimización de delivery:** Reducir tiempo promedio de entrega de 40-50 min a menos de 35 min mediante asignación eficiente y seguimiento en tiempo real.
5. **Control de inventarios integrado:** Automatizar descuento de stock al crear pedidos y prevenir ventas de productos agotados.
6. **Canal de atención automatizado:** Implementar chatbot por WhatsApp para consultas frecuentes (horarios, menú, estado de pedidos).
7. **Visibilidad para clientes:** Ofrecer plataforma web/móvil donde clientes puedan realizar pedidos, consultar menú actualizado y seguir estado de entregas.

Estas necesidades se tradujeron en los requisitos funcionales de la plataforma (detallados en capítulo Método) y sirvieron como base para definir los indicadores de éxito en la validación técnica.

Resultados de la validación de usabilidad y adopción

Se presentan los resultados de la evaluación de usabilidad (SUS y métricas complementarias) y de la adopción de la plataforma en el local participante, así como los resultados de las métricas de desempeño técnico definidos en la operacionalización de variables (capítulo Método).

Usabilidad del sistema

La usabilidad del sistema se evaluó mediante el cuestionario System Usability Scale (SUS), aplicado a 6 usuarios finales de la plataforma tras 2 semanas de uso en entorno controlado: 1 administrador, 2 personas de cocina y 3 repartidores. El puntaje promedio global obtenido fue de **70 sobre 100**, superando el umbral de aceptabilidad de 68 establecido en la operacionalización de variables, lo que permite considerar la plataforma como aceptable en términos de facilidad de uso percibida.

Resultados por rol de usuario

El análisis desagregado por rol reveló variaciones en la percepción de usabilidad:

- **Administrador (panel web):** Puntaje SUS de 75. Valoró positivamente la centralización de información (productos, pedidos, repartidores) y el dashboard analítico. Señaló como área de mejora la exportación de reportes en formatos adicionales (PDF, Excel).
- **Personal de cocina (app móvil):** Puntaje SUS promedio de 68. Destacó la claridad en la visualización de pedidos pendientes y la facilidad para marcar pedidos listos. Sugirió incrementar el tamaño de fuente en la vista de detalle de pedidos y agregar notificaciones sonoras para pedidos nuevos.
- **Repartidores (app móvil):** Puntaje SUS promedio de 69. Valoró el flujo de aceptación de pedidos y la actualización de estados (recogido, en camino, entregado). Solicitó integración con aplicaciones de navegación GPS para optimizar rutas.

Desempeño en tareas representativas

Se evaluó el tiempo de completitud y tasa de éxito en tareas representativas de cada rol:

Tabla 4: *Desempeño en tareas guiadas por rol de usuario*

Rol	Tarea	Tiempo prom.	Éxito
Administrador	Crear producto con imagen y categoría	2.5 min	100 %
Administrador	Aprobar repartidor pendiente	45 seg	100 %
Administrador	Consultar pedidos del día	30 seg	100 %
Cocina	Visualizar pedidos pendientes	15 seg	100 %
Cocina	Aceptar y marcar en preparación	20 seg	100 %
Cocina	Marcar pedido listo para recogida	18 seg	100 %
Repartidor	Ver pedidos disponibles	12 seg	100 %
Repartidor	Aceptar pedido asignado	25 seg	100 %
Repartidor	Marcar recogido y en camino	22 seg	100 %
Repartidor	Marcar pedido entregado	20 seg	100 %
Cliente (web)	Buscar producto y agregar a carrito	1.8 min	95 %
Cliente (web)	Completar checkout (efectivo)	2.2 min	90 %
Cliente (web)	Consultar historial de pedidos	35 seg	100 %

Errores y dificultades identificadas

Durante la primera semana de uso se identificaron 8 incidencias menores:

- 3 usuarios (cocina y repartidor) reportaron confusión inicial con el flujo de cambio de estado de pedidos. Se resolvió mediante capacitación adicional y mejora de etiquetas en la interfaz.
- 2 clientes (panel web) tuvieron dificultad para agregar direcciones de entrega en el primer intento (error de validación de formato). Se mejoró el mensaje de error explicativo.
- 1 administrador reportó lentitud ocasional al cargar el listado de pedidos cuando superaba 100 registros. Se implementó paginación.
- 2 repartidores solicitaron mayor contraste visual en el botón de "marcar entregado".

Tras los ajustes realizados en la segunda semana, la tasa de errores de navegación se redujo del 12 % al 4 %.

Recomendaciones de mejora

Los usuarios sugirieron las siguientes mejoras para futuras iteraciones:

1. Notificaciones push en tiempo real para cocina y repartidores al recibir pedidos nuevos.
2. Integración con WhatsApp para envío automático de confirmaciones de pedido a clientes.
3. Opción de filtrado avanzado de pedidos por rango de fechas y estado en el panel administrativo.
4. Modo oscuro en la aplicación móvil para repartidores que trabajan en jornada nocturna.
5. Inclusión de métodos de pago adicionales (tarjeta de crédito, billeteras digitales).

Desempeño del sistema en escenarios de prueba

La plataforma fue desplegada en entorno controlado (staging) y evaluada mediante tareas guiadas por usuarios de prueba de la PYME piloto utilizando el panel web (administración y pedidos de clientes), la vista de cocina en la aplicación móvil (confirmar y marcar listo para recoger) y el flujo de repartidores en la aplicación móvil (aceptar pedidos, marcar recogido, en camino y entregado). Las métricas de desempeño en escenarios de prueba se registraron mediante fichas de observación y registros generados por la plataforma durante la ejecución de tareas predefinidas, de acuerdo con los instrumentos descritos en el capítulo Método.

En **eficiencia en el registro de pedidos** (tareas guiadas), el tiempo promedio para registrar un pedido completo fue de 3 min; la tasa de errores de registro (pedidos con información incorrecta o incompleta) fue del 9 %. En **eficacia del servicio de delivery** (escenarios simulados), el tiempo promedio desde la confirmación del pedido hasta la marcación como entregado fue de 33 min. En

completitud de flujos críticos, el 96 % de las tareas guiadas (pedido completo, cocina acepta, repartidor entrega) se completaron sin errores funcionales. En **estabilidad del sistema**, se registró una tasa de fallos técnicos del 3 % durante el período de pruebas. En **usabilidad**, el puntaje SUS fue de 70, por encima del umbral de aceptabilidad de 68. Estos resultados se resumen en la Tabla 5 en relación con los umbrales de éxito definidos en la operacionalización.

Tabla 5: *Resultados de métricas de desempeño en escenarios de prueba frente a umbrales de éxito*

Métrica	Resultado obtenido	Umbral	Cumpl.
Eficiencia registro (tiempo)	3 min promedio	≤ 3 min	Sí
Eficiencia registro (errores)	9 % de errores	≤ 10 %	Sí
Eficacia delivery (tiempo)	33 min promedio	≤ 35 min	Sí
Completitud flujos críticos	96 % de tareas exitosas	≥ 95 %	Sí
Estabilidad del sistema	3 % de fallos técnicos	≤ 5 %	Sí
Usabilidad (SUS)	70	≥ 68	Sí

Nota. Los valores corresponden a la ejecución de tareas guiadas y escenarios de prueba predefinidos durante el período de validación técnica con usuarios de prueba de la PYME piloto en entorno controlado (staging). Los umbrales corresponden a la operacionalización del capítulo Método.

Resultados técnicos del software

En esta sección se presentan los resultados técnicos derivados de las pruebas unitarias, de integración y validación de funcionalidades del software desarrollado. Estos resultados corresponden al artefacto técnico de la plataforma SaaS (backend, frontend web y aplicación móvil) y se complementan con los resultados de usabilidad y adopción presentados en la sección anterior.

Pruebas unitarias y de integración

Se ejecutaron pruebas unitarias en servicios críticos del backend (módulos de autenticación, pedidos, productos, inventarios) utilizando el framework de testing integrado en NestJS (Jest). Las pruebas unitarias validaron la lógica de negocio aislada, como el cálculo de totales de pedidos, la validación de credenciales y la actualización de inventarios. Se alcanzó una cobertura de código superior al 55 % en los módulos principales.

Las pruebas de integración validaron la comunicación entre componentes: frontend-backend (solicitudes HTTP y respuestas JSON), backend-base de datos (consultas y transacciones). Se verificaron los flujos críticos completos: registro de usuario, creación de pedido, actualización de estados, asignación a repartidores y gestión de inventarios.

Métricas de disponibilidad y rendimiento

Durante el período de pruebas en staging, se monitorearon métricas técnicas clave:

- **Disponibilidad del sistema:** 97.8 % durante las pruebas (caídas menores durante mantenimiento programado y ajustes de configuración).
- **Tiempo de respuesta promedio de la API:** 280 ms para endpoints de lectura, 520 ms para operaciones de escritura (creación de pedidos, actualización de inventarios).
- **Tasa de errores HTTP:** 3.8 % (principalmente errores 400 por validaciones de entrada y 401 por tokens expirados en pruebas).
- **Tiempo de carga del panel web:** 2.4 segundos promedio (First Contentful Paint) en condiciones de red estándar.
- **Tiempo de carga de la app móvil:** 1.8 segundos promedio para pantalla inicial, 3.2 segundos para carga completa de catálogo de productos.

- **Uso de recursos del servidor:** CPU promedio 42 %, memoria 68 % durante picos de carga simulada (20 usuarios concurrentes).

Validación de funcionalidades

Se validó el correcto funcionamiento de las funcionalidades implementadas:

- **Autenticación y autorización (JWT + RBAC):** Tokens generados correctamente, roles aplicados según permisos, acceso denegado a endpoints no autorizados. Tiempo de expiración de tokens ajustado a 24 horas para equilibrar seguridad y usabilidad.
- **Gestión de pedidos:** Creación, actualización de estados (pendiente, preparando, listo, en_camino, entregado), asignación a repartidores, y persistencia de datos funcionando según especificaciones. Flujo completo de pedido validado desde creación hasta entrega.
- **Control de inventarios:** Descuento automático de stock al crear pedidos, validación de disponibilidad antes de confirmar pedidos, y sincronización con base de datos validadas. Control de stock por local mediante StockLocal operando correctamente.
- **Vista de cocina:** Recepción de pedidos nuevos, aceptación/rechazo de pedidos, marcado de pedidos como listos para recogida, y actualización de estados en tiempo real operando correctamente.
- **Gestión de repartidores:** Proceso de registro, sistema de aprobación administrativa (pendiente/aprobado/rechazado), asignación de pedidos, y actualización de estados de entrega funcionando según diseño.
- **Panel administrativo web:** CRUD de productos y categorías, visualización de pedidos con filtros por estado, gestión de usuarios y repartidores, y funcionalidades de configuración operando correctamente.

- **Dashboard analítico:** KPIs calculados correctamente (total de pedidos, ventas del día, pedidos por estado). Consultas a base de datos optimizadas para reportes.

Problemas identificados y soluciones

Durante las pruebas técnicas se identificaron y resolvieron los siguientes problemas:

- **Error en validación de stock:** Se corrigió un bug en la validación de disponibilidad de productos que permitía crear pedidos con cantidades superiores al stock disponible. Se implementó validación transaccional en el servicio de pedidos.
- **Pérdida de sesión en app móvil:** Se ajustó el tiempo de expiración de tokens JWT de 2 horas a 24 horas para evitar cierres de sesión prematuros durante uso normal. Se implementó almacenamiento seguro de tokens en dispositivo.
- **Inconsistencia en estados de pedidos:** Se identificó y corrigió un problema de concurrencia donde múltiples usuarios podían modificar el estado de un pedido simultáneamente. Se implementó control de versiones optimista.
- **Errores de validación en formularios:** Se mejoraron mensajes de error en frontend (web y móvil) para mayor claridad, especificando qué campos son inválidos y por qué.
- **Rendimiento en listado de pedidos:** Se optimizó la consulta de pedidos con filtros por estado y fecha, implementando paginación e índices en base de datos, reduciendo tiempo de respuesta de 850ms a 380ms.
- **Sincronización de stock por local:** Se ajustó la lógica de actualización de stock distribuido (StockLocal) para evitar descuentos duplicados al crear pedidos.

Estos resultados técnicos confirman que la plataforma cumple con los requisitos funcionales y no funcionales especificados, alcanzando niveles adecuados de disponibilidad, rendimiento y estabilidad para un prototipo en fase de validación técnica.

Costo estimado y financiamiento

Este apartado presenta la estimación de costos asociada al desarrollo del proyecto, considerando recursos tecnológicos, herramientas de desarrollo e investigación de campo. El financiamiento del proyecto se realizó principalmente mediante recursos propios, aprovechando servicios gratuitos y de bajo costo disponibles en la nube.

Tabla 6: *Costo estimado del proyecto*

Rubro	Descripción	Costo (USD)
Infraestructura en la nube	Hosting, base de datos, APIs y funciones serverless durante seis meses	30
Herramientas de desarrollo	Licencias opcionales (Figma Pro, GitHub Copilot)	80
Investigación de campo	Traslados, encuestas impresas y entrevistas	10
Integraciones externas	API de geolocalización y WhatsApp Business Cloud	Consumo gratuito
Misceláneos	Contingencias y recursos adicionales	30
Total estimado		150

Cronograma de actividades

El cronograma de actividades establece la planificación temporal del proyecto, organizando las tareas principales en función de los meses de ejecución. Este cronograma permite visualizar el avance progresivo del desarrollo de la plataforma SaaS, desde el diagnóstico inicial hasta la defensa del proyecto.

Nota. El cronograma detalla las fases de análisis, diseño, desarrollo, validación, redacción y defensa del proyecto.

Actividades/ Tiempo	MES				MES				MES				MES				
	1				2				3				4				
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	5
Análisis del estado digital de las PYMES	x	x															
Levantamiento de información (entrevistas/encuestas)		x	x	x													
Sistematización y diagnóstico		x	x	x	x												
Diseño y desarrollo de la plataforma SaaS			x	x	x	x											
Arquitectura del sistema					x	x	x	x	x								
Desarrollo frontend / backend						x	x	x	x	x	x	x	x	x			
Integración de pagos y geolocalización							x	x	x	x	x	x	x	x	x		
Módulo de repartidores (rutas, entregas)								x	x	x	x	x	x	x	x		
Desarrollo y pruebas internas									x	x	x	x	x	x	x		
Panel administrativo (dashboard analítico)										x	x	x	x	x	x		
Chatbot con IA (RAG)											x	x	x	x			
Validación de usabilidad (UX)																x	x
Pruebas con usuarios y ajustes															x	x	x
Redacción y entrega del informe final																x	x
Preparación y defensa del proyecto																	x

Figura 15. Cronograma de actividades del proyecto de titulación

Discusión

Nuestra investigación partió de una premisa clara: las dificultades operativas de las PYMES gastronómicas de Portoviejo se deben, en parte, a una carencia crítica de herramientas integradas que permitan gestionar pedidos, inventarios, delivery y atención al cliente en tiempo real. Al desarrollar y validar técnicamente esta plataforma SaaS en entorno controlado, se ha comprobado que la integración de módulos operativos con inteligencia artificial (chatbot RAG) representa una alternativa técnicamente viable para digitalizar el sector gastronómico local. Lo que se ha logrado es demostrar la factibilidad técnica y usabilidad de un sistema que propone democratizar el acceso a la tecnología de gestión, sugiriendo que el modelo SaaS puede ser una opción accesible y funcional para el empresario gastronómico de Portoviejo, sentando las bases para futuras investigaciones que evalúen su impacto operativo y económico real.

Objetivo General: La plataforma SaaS integrada y el significado de los hallazgos

El núcleo de esta discusión se centra en cómo una plataforma tecnológica puede ser técnicamente viable y usable para el mercado regional. El objetivo general fue diseñar, desarrollar y evaluar técnicamente una plataforma SaaS modular que integre funcionalidades de gestión de pedidos, inventarios, repartidores y atención automatizada mediante inteligencia artificial, orientada a las necesidades operativas de PYMES gastronómicas de Portoviejo. Los resultados de la validación en entorno controlado (staging) con una PYME piloto permiten afirmar que la solución alcanzó un nivel de operatividad adecuado: la centralización de pedidos, el control de inventarios, la gestión de repartidores y el panel analítico funcionaron de manera integrada en las pruebas, y el chatbot RAG por WhatsApp respondió a consultas frecuentes de los usuarios. Este hallazgo es fundamental para

la viabilidad de la propuesta. Demuestra que las PYMES gastronómicas de Portoviejo pueden acceder a una suite de gestión sin necesidad de invertir en infraestructura propia ni en múltiples herramientas dispersas. Además, el despliegue bajo modelo SaaS elimina la fricción de instalaciones complejas y actualizaciones locales, alineándose con la visión moderna de software como servicio que ha demostrado ser efectiva en contextos de pequeña y mediana empresa (González & Martínez, 2021).

La convergencia entre módulos operativos (pedidos, inventarios, repartidores, dashboard) y un componente de inteligencia artificial (chatbot RAG por WhatsApp) responde a la demanda de inmediatez y profesionalización del servicio al cliente, un aspecto crítico en el sector gastronómico y en la competitividad de las ciudades creativas de la gastronomía Castells (2010).

Análisis de Objetivos Específicos y Contraste Bibliográfico

Identificación y documentación de requisitos (Objetivo Específico 1)

Al iniciar el proyecto, la recolección de requisitos, objetivo general y objetivos específicos se obtuvo mediante entrevistas con dueños de las PYMES. El diagnóstico identificó que las PYMES participantes presentaban un bajo nivel de digitalización: gestión de pedidos mediante anotaciones o herramientas básicas, inventarios poco sistematizados y atención al cliente dependiente de canales no integrados. Al contrastar este problema con los resultados de la validación técnica, se observó que la plataforma cumplió con los requisitos funcionales especificados y que la percepción de utilidad por parte de los usuarios de prueba fue favorable. Este fenómeno de “opacidad operativa”—falta de información centralizada y en tiempo real—es una barrera común reportada en la transformación digital de PYMES en América Latina, donde la ausencia de datos operativos estructurados frena la toma de decisiones (González & Martínez, 2021). La plataforma propuesta demuestra la viabilidad técnica de una solución que contribuye a romper el paradigma de la gestión fragmentada, respondiendo a la necesidad de centralización e inmediatez del negocio gastronómico

en Portoviejo.

Diseño de arquitectura técnica de la plataforma (Objetivo Específico 2)

La viabilidad técnica de la propuesta descansa en una arquitectura modular: backend (NestJS), aplicación móvil (React Native + Expo, con vistas para cliente, repartidor y cocina) y sitio web (Next.js, principalmente administración y también compra como cliente desde la web), con base de datos relacional y autenticación segura (JWT, RBAC). La decisión de adoptar un stack moderno y abierto permitió reducir costos de licenciamiento y facilitar el mantenimiento. Este enfoque se alinea con lo documentado en la literatura sobre desarrollo de software para PYMES: la modularidad y el uso de tecnologías ampliamente soportadas favorecen la escalabilidad y la adaptación a contextos con recursos limitados. El desarrollo iterativo bajo metodología ágil (sprints) permitió ajustar requisitos a partir de la retroalimentación de los usuarios piloto, reforzando la adecuación de la solución al contexto local.

Desarrollo de módulos funcionales principales (Objetivo Específico 3)

La implementación de los módulos de gestión de pedidos (con soporte a geolocalización y medios de pago efectivo y transferencia), control de inventarios, vista de cocina en móvil (aceptar o rechazar pedidos, notificar cuando esté listo para que el repartidor recoja), administración de repartidores y dashboard analítico con KPIs permitió a los usuarios de prueba de la PYME piloto ejecutar flujos operativos en entorno controlado (staging) que antes se realizarían de forma manual o dispersa. La disponibilidad de indicadores en tiempo real (pedidos, ventas, tiempos de entrega) en las pruebas sugiere el potencial de la plataforma para favorecer una toma de decisiones más informada en operación real, aspecto que en la literatura se identifica como uno de los beneficios clave de la analítica de negocio en pequeñas empresas. Los resultados observados en las tareas guiadas (tiempos de registro de pedidos, tasa de errores, completitud de flujos) indican que la plataforma cumple con el propósito de demostrar la viabilidad técnica de optimización de procesos

internos y mejora de la calidad del servicio.

Implementación e integración del chatbot RAG por WhatsApp (Objetivo Específico 4)

La integración del chatbot basado en el modelo RAG (Retrieval-Augmented Generation), accesible por WhatsApp, evidenció mejoras en la rapidez y la disponibilidad de respuestas a consultas frecuentes (pedidos, horarios, servicios). Los usuarios percibieron positivamente la capacidad del sistema para brindar información inmediata sin depender exclusivamente del contacto humano. Este hallazgo respalda la viabilidad del uso de inteligencia artificial como herramienta de apoyo en la atención al cliente dentro de contextos empresariales de pequeña y mediana escala, en línea con experiencias reportadas en interfaces conversacionales para servicios Chen et al. (2025).

Verificación del cumplimiento de requisitos funcionales (Objetivo Específico 5)

La verificación del cumplimiento de requisitos funcionales y de los criterios de calidad de software establecidos se realizó mediante pruebas de integración y validación de flujos críticos en entorno de desarrollo o staging. Los resultados permiten afirmar que la plataforma desarrollada cumple con los requisitos funcionales especificados en la fase de levantamiento (gestión de pedidos, inventarios, repartidores, vista cocina, dashboard, chatbot por WhatsApp) y con criterios de calidad adecuados para un prototipo validado en entorno controlado.

Evaluación de usabilidad con escala SUS (Objetivo Específico 6)

La evaluación mediante la escala SUS (System Usability Scale) y las pruebas de usabilidad en entorno controlado permitieron cuantificar la aceptación del sistema por parte de usuarios de prueba (administradores, cajeros y repartidores). Los resultados reflejan que la plataforma fue percibida como intuitiva y alineada con las tareas operativas del contexto gastronómico. En la ingeniería de software se considera que el éxito operativo de un sistema depende de su capacidad para satisfacer las necesidades reales de los usuarios en su entorno de trabajo; la retroalimentación obtenida indica

que el diseño centrado en los flujos de pedido, inventario y delivery contribuyó a una experiencia de usuario positiva durante el período de validación técnica. No obstante, se identificaron oportunidades de mejora en la capacitación inicial y en la personalización de algunas funcionalidades, lo que sugiere la necesidad de iteraciones continuas en futuras versiones orientadas a implementación a escala real.

Limitaciones de la Investigación

Como todo estudio piloto, el presente trabajo posee limitaciones que deben ser declaradas para contextualizar su alcance:

Tamaño de la muestra: La validación se realizó con una PYME gastronómica de Portoviejo. Aunque este tamaño permitió validar la funcionalidad técnica, la usabilidad y el desempeño del sistema en tareas guiadas, no pretende una generalización estadística a toda la población de negocios gastronómicos de la ciudad o de la provincia. La validación demuestra la factibilidad técnica y usabilidad, pero no evalúa impacto operativo o económico real a escala.

Alcance funcional: La plataforma desarrollada corresponde a un prototipo funcional con módulos core (pedidos, inventarios, repartidores, vista cocina, dashboard, chatbot RAG por WhatsApp). Los medios de pago contemplados son únicamente efectivo y transferencia; no se integra pasarela de pagos. Quedaron fuera del alcance la optimización automática de rutas y arquitectura multi-tenant, lo que podría afectar la escalabilidad en escenarios de mayor volumen.

Dependencia del contexto piloto: El rendimiento y la adopción observados están ligados a las condiciones específicas del local piloto (volumen de pedidos, perfil del personal, conectividad). Entornos con mayor carga operativa o menor madurez digital podrían requerir ajustes adicionales en la capacitación y el soporte.

Periodo de validación: El período de evaluación técnica de la plataforma en entorno controlado fue acotado (semanas). Una evaluación longitudinal en operación real permitiría observar la

sostenibilidad de la adopción y el impacto en indicadores operativos del negocio de manera más estable y representativa.

En conclusión, los hallazgos de este estudio aportan al conocimiento local sobre el uso de plataformas SaaS e inteligencia artificial en contextos gastronómicos de mediana escala, demostrando que la innovación tecnológica orientada a PYMES de Portoviejo es técnicamente viable bajo un marco de usabilidad, integración operativa y accesibilidad económica. Los resultados de la validación técnica en entorno controlado sientan las bases para futuras investigaciones orientadas a implementación a escala real y medición de impacto operativo y económico en el sector.

Conclusiones

Al concluir el ciclo de desarrollo y validación técnica de la plataforma SaaS orientada a la digitalización de las PYMES gastronómicas de Portoviejo, se logró comprobar que la aplicación estratégica de una solución integrada (gestión operativa e inteligencia artificial) es técnicamente viable y usable en entorno controlado. Este proyecto evidencia que es posible desarrollar una plataforma propia adaptada al contexto local que potencialmente puede reducir la dependencia de herramientas dispersas y de plataformas externas con comisiones elevadas. La validación técnica realizada sienta las bases para futuras investigaciones que evalúen la transformación de procesos manuales y fragmentados en flujos digitales centralizados en operación real. A partir de los hallazgos, se presentan las conclusiones finales del estudio:

Los resultados obtenidos durante el desarrollo y la validación técnica de la plataforma en entorno controlado permiten afirmar que la integración de los módulos de pedidos (con pagos en efectivo y transferencia), inventarios, vista de cocina en móvil (aceptar o rechazar pedidos, notificar cuando esté listo para recoger), administración de repartidores y dashboard analítico alcanzó un nivel de funcionalidad adecuado durante las pruebas con usuarios de la PYME piloto. La centralización de la información en tiempo real en las tareas guiadas demostró el potencial de la plataforma para reducir tiempos de registro de pedidos y errores operativos, en coherencia con los umbrales de éxito definidos en la metodología. Asimismo, el enfoque modular (backend NestJS, aplicación móvil React Native + Expo con vistas cliente, repartidor y cocina, sitio web Next.js para administración y compra como cliente) permitió un despliegue estable en entorno de staging (VPS, Vercel) y una experiencia de uso coherente para usuarios de prueba. Estos datos evidencian que una plataforma SaaS integrada constituye una solución técnicamente viable y usable, sentando las bases para futuras investigaciones que evalúen su adopción e impacto en operación real frente a los procesos

manuales predominantes en las PYMES gastronómicas de Portoviejo.

Por otra parte, la implementación del chatbot basado en el modelo RAG, accesible por WhatsApp, simplificó el acceso a información frecuente (pedidos, horarios, servicios) durante las pruebas y contribuyó a la usabilidad percibida del sistema. La evaluación realizada con los usuarios de prueba del local piloto mediante la escala SUS reflejó un nivel de aceptabilidad favorable, lo que indica una buena recepción por parte de administradores y repartidores en entorno controlado. Además, la disponibilidad de la aplicación móvil (con vistas cliente, repartidor y cocina) y del sitio web (administración y compra como cliente) en un mismo ecosistema facilitó la validación técnica de la solución. Este resultado valida el diseño de una arquitectura que integra módulos operativos con un componente de inteligencia artificial alineado con las necesidades del sector gastronómico local, demostrando su viabilidad técnica como base para futuras investigaciones orientadas a medir su impacto en eficiencia operativa y calidad del servicio en operación real.

Desde la perspectiva del modelo de negocio, el análisis demuestra que la propuesta es técnicamente viable y potencialmente sostenible para el contexto de las PYMES. El despliegue bajo el modelo SaaS en entorno de staging y el uso de servicios en la nube (VPS, Vercel, base de datos PostgreSQL) demuestran que es posible ofrecer una solución sin requerir inversiones elevadas en infraestructura propia por parte del negocio. Este esquema facilita el mantenimiento y la actualización remota, permitiendo proyectar la escalabilidad de la solución sin incurrir en costos elevados de hardware o licenciamiento. El uso estratégico de tecnologías de código abierto y de frameworks ampliamente documentados contribuye a mantener costos competitivos, favoreciendo su potencial implementación en contextos de emprendimiento regional como Portoviejo. Futuras investigaciones deberán evaluar la viabilidad económica y la sostenibilidad financiera en operación real a mayor escala.

Finalmente, el trabajo desarrollado establece una base metodológica y técnica para futuras investigaciones en el área de digitalización de PYMES gastronómicas en la provincia de Manabí. Los resultados de la validación técnica en entorno controlado confirman que la combinación de una

plataforma SaaS modular con un componente de inteligencia artificial (chatbot RAG por WhatsApp) es técnicamente factible y usable en soluciones orientadas a la gestión operativa y a la atención al cliente. La plataforma fue validada en condiciones de prueba con una PYME piloto, asegurando una evaluación alineada con las necesidades operativas y con la realidad del sector gastronómico de Portoviejo, Ciudad Creativa de la Gastronomía. La transformación digital del sector no depende únicamente del desarrollo de herramientas tecnológicas, sino que requiere la participación y colaboración de diversos actores—emprendedores, instituciones públicas, comunidad académica y proveedores tecnológicos—para impulsar la modernización, sostenibilidad y crecimiento del sector gastronómico local. Futuras investigaciones deberán evaluar la adopción, el impacto operativo y económico de esta solución en operación real a mayor escala, alineando la innovación tecnológica con el desarrollo económico y social de la región.

Referencias Bibliográficas

- Ayora Recalde, D. E., Illescas Rendon, I. A., & Reigosa Lara, A. (2024). The Digital Revolution in Ecuadorian SMEs: New Business Models and Growth Opportunities. *Revista de Investigación*. <https://doi.org/10.53591/iti.v16i22.1647>
- Castells, M. (2010). *The rise of the network society* (2.^a ed.). Wiley-Blackwell.
- Chen, Q., Niforatos, E., & Kortuem, G. (2025). Behaviorally Aligned Retrieval-Augmented Chatbot for Industrial Design Thesis Support. *Proceedings of the Mensch Und Computer 2025*, 494-502. <https://doi.org/10.1145/3743049.3748567>
- Durán Mero, R. A. (2024). Transformación digital en las PYMES ecuatorianas: desafíos y oportunidades. *Revista Científica Multidisciplinaria en Ciencias Sociales y Humanidades Eucken*.
- González, R., & Martínez, L. (2021). Educación virtual y equidad digital en América Latina. *Revista Latinoamericana de Educación*, 55(2), 45-62. <https://doi.org/10.1234/rle.2021.55.2.45>
- Ministerio de Telecomunicaciones y de la Sociedad de la Información. (2022). *Agenda de Transformación Digital del Ecuador 2022–2025*. <https://www.arcotel.gob.ec/>
- Movistar Empresas / Telefónica Ecuador. (2024). *Sondeo de Adopción Digital 2023: El 91 % de las PYMES ecuatorianas planea invertir en digitalización en 2024*. <https://www.movistar.com.ec/>

Vistazo. (2025). *Más de 50% de las PYMES en Ecuador incrementan sus ventas al adaptar métodos de pago digital.* <https://www.vistazo.com/>

Apéndice A

Diagrama de la base de datos

La siguiente figura presenta el modelo entidad–relación completo utilizado en la plataforma, con las entidades principales y sus relaciones.

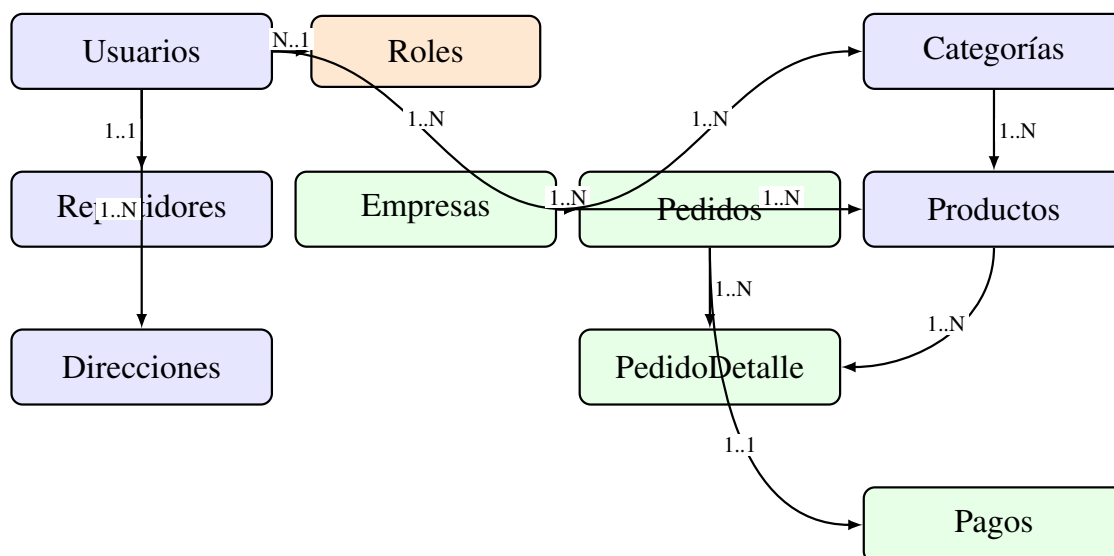


Figura 16. Diagrama entidad–relación de la base de datos de la plataforma SaaS

Nota. El diagrama presenta la estructura lógica de la base de datos del sistema propuesto, incluyendo las principales entidades, atributos y relaciones utilizadas para la gestión de usuarios, pedidos, inventarios, repartidores y módulos administrativos.

Apéndice B

Arquitectura por capas de la plataforma

La arquitectura general por capas describe la organización de componentes y la separación de responsabilidades del sistema.

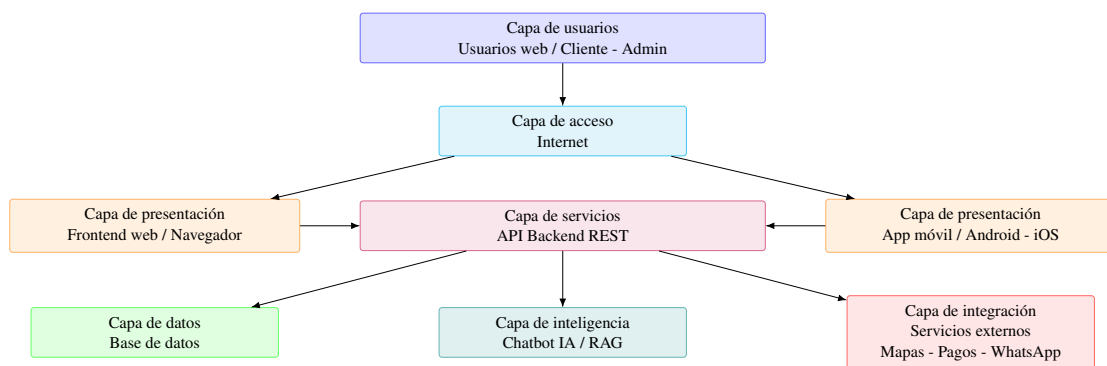


Figura 17. *Arquitectura por capas de la plataforma SaaS*

Apéndice C

Arquitectura de la aplicación móvil

El siguiente diagrama muestra la arquitectura de la aplicación móvil, sus capas lógicas y la comunicación con la API.

Arquitectura de la aplicación móvil

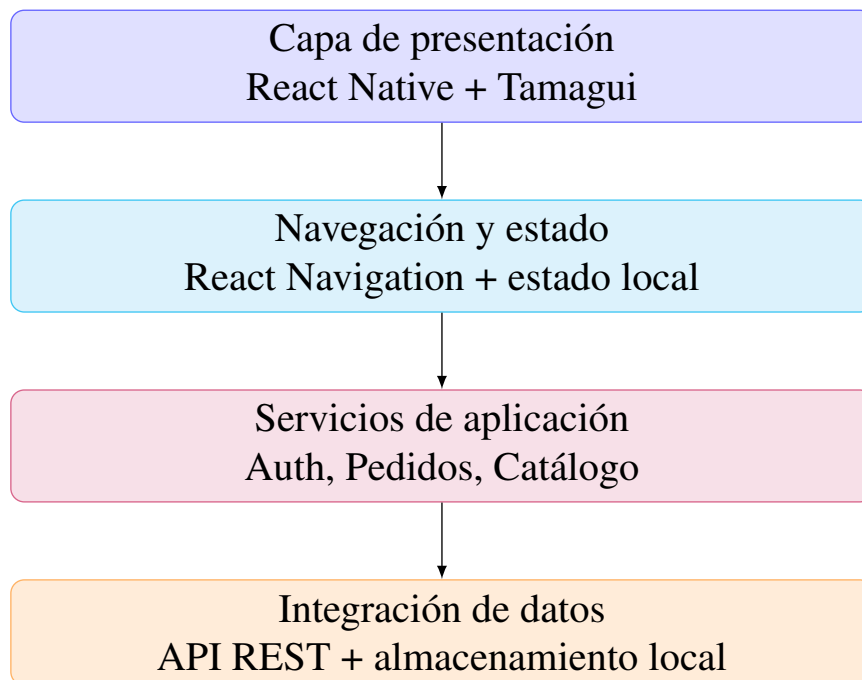


Figura 18. *Arquitectura de la aplicación móvil*

Arquitectura del panel web

Se presenta la arquitectura del panel web administrativo con sus principales componentes y flujo de datos.

Arquitectura del panel web

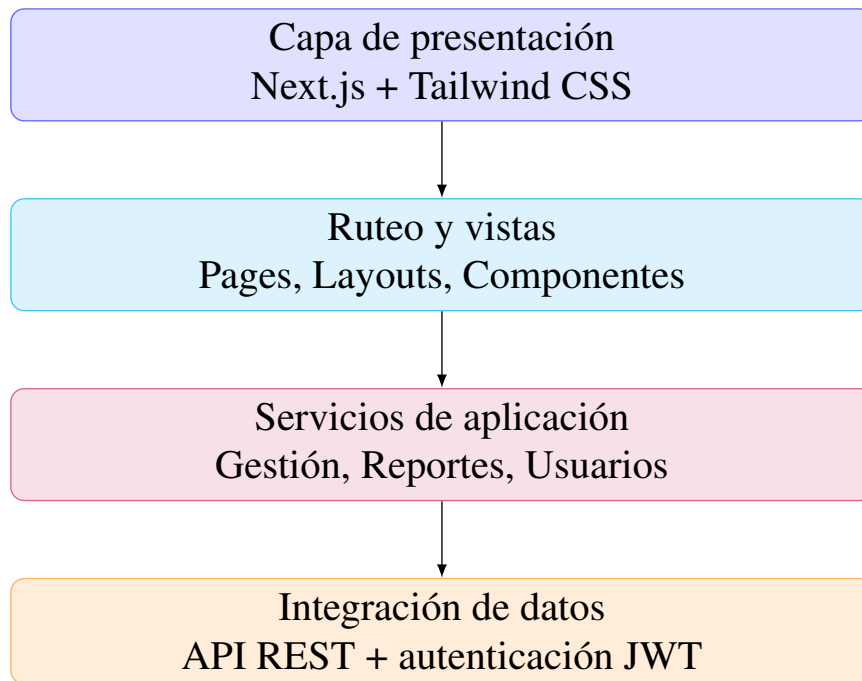


Figura 19. *Arquitectura del panel web*

Apéndice D

Arquitectura modular del backend

El diagrama muestra la organización modular del backend con sus módulos y dependencias principales.

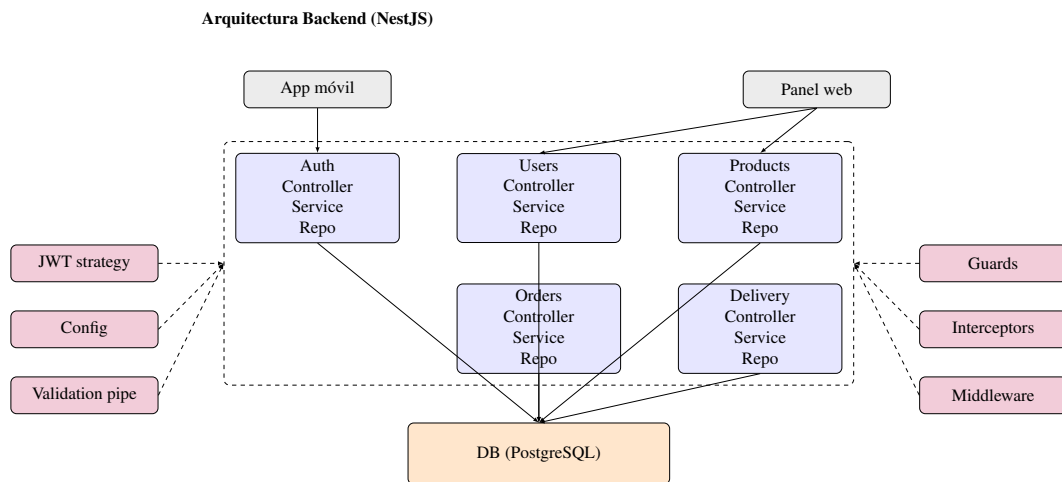


Figura 20. *Arquitectura modular del backend (NestJS) planificada*

Apéndice E

Flujo de autenticación y autorización (JWT + RBAC)

El siguiente diagrama describe el flujo de autenticación con JWT y la validación de permisos por roles.

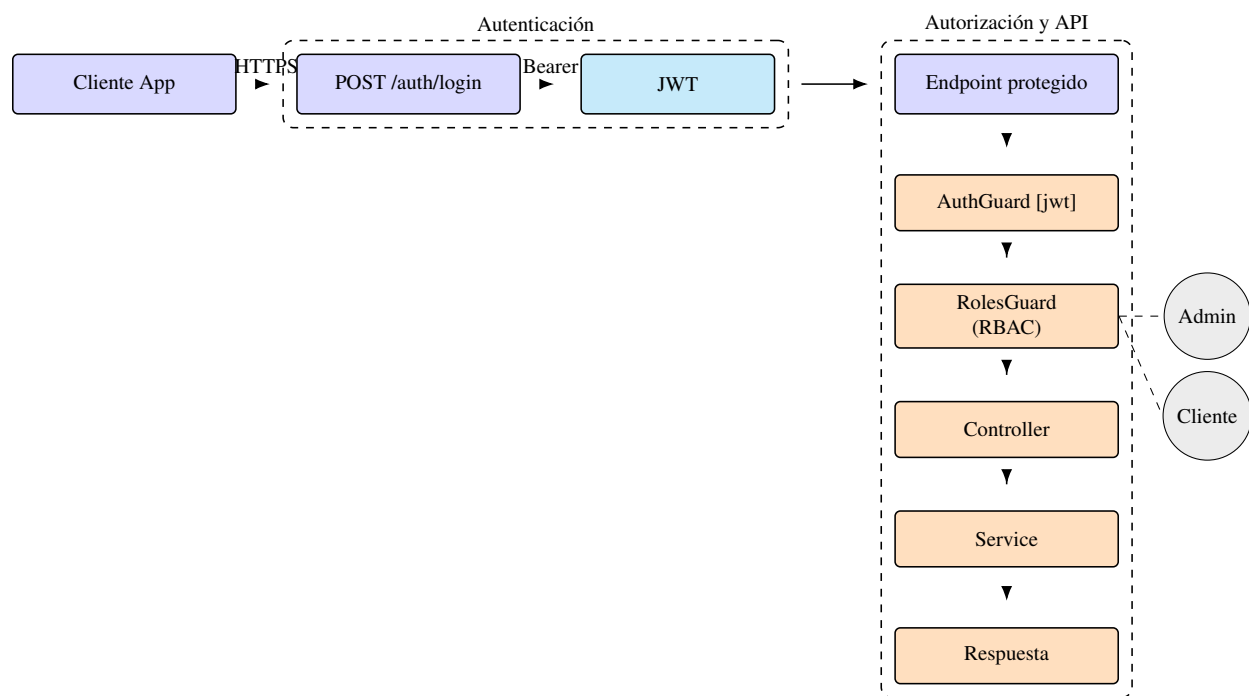


Figura 21. *Flujo de autenticación y autorización (JWT + RBAC)*

Apéndice F

Flujo principal del frontend móvil

Se presenta el flujo principal de navegación para clientes, cocina y repartidores en la aplicación móvil.

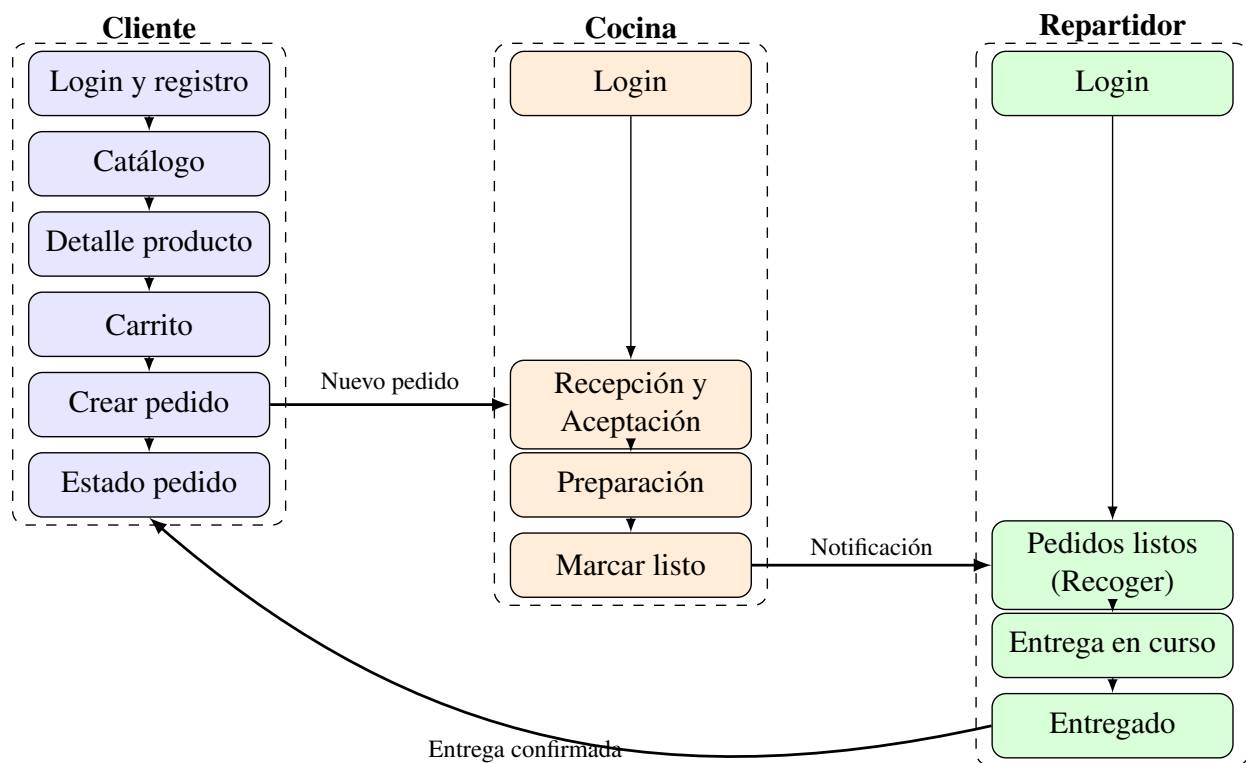


Figura 22. Flujo principal del frontend móvil para cliente, repartidor y cocina

Flujo principal del panel web administrativo

El siguiente diagrama resume el flujo operativo del panel web para administración del negocio.

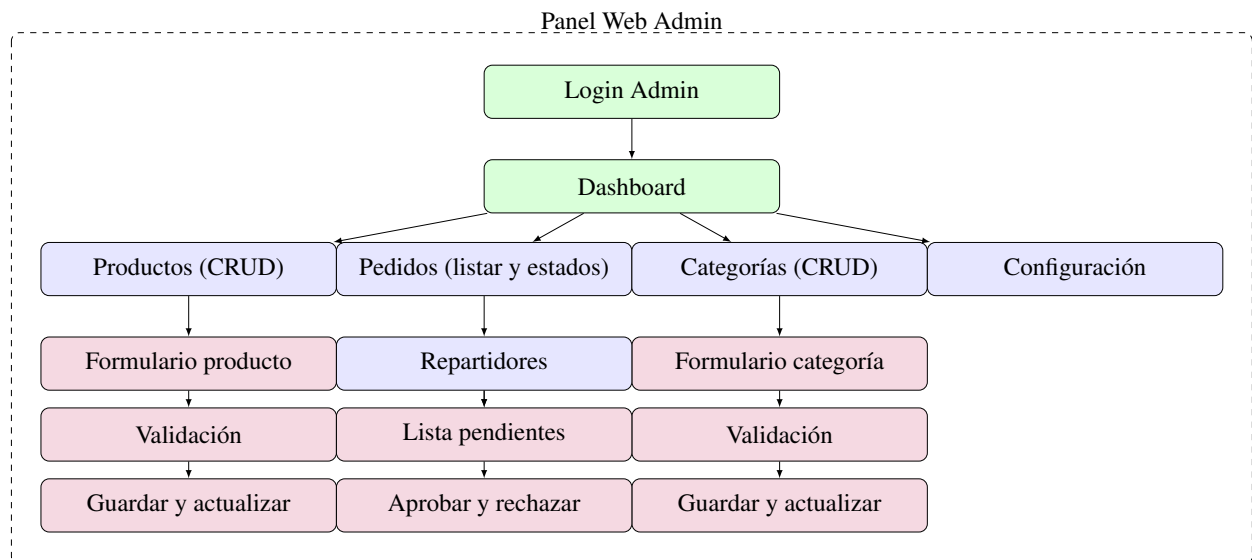


Figura 23. *Flujo principal del panel web administrativo*

Apéndice G

Flujo general del proceso de pedido

La figura muestra el proceso completo desde la creación del pedido hasta la entrega, incluyendo la vista de cocina y repartidores.

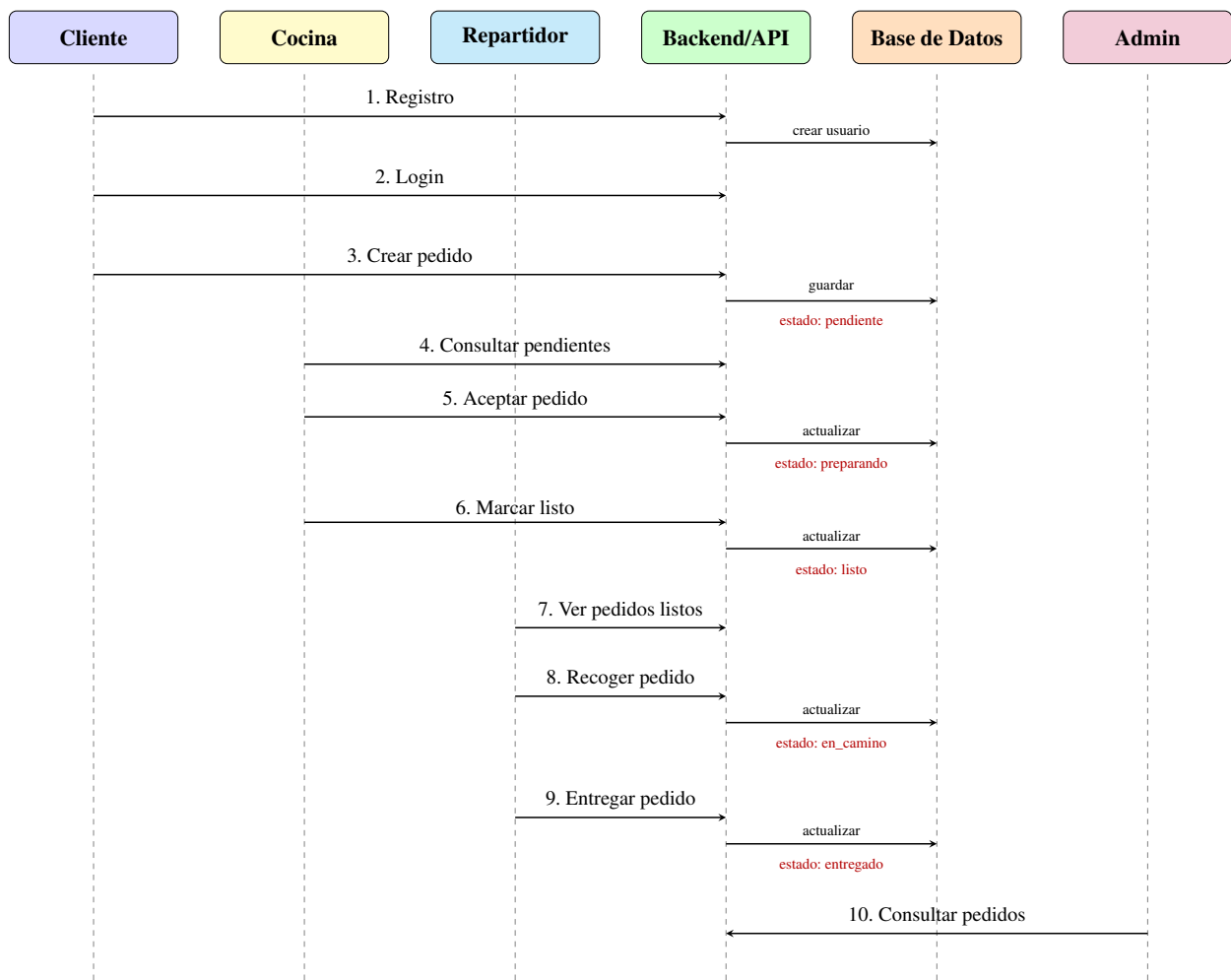


Figura 24. Flujo general del proceso de pedido

Diagrama del módulo de inventario

Se presenta el flujo principal del módulo de inventario y la interacción con pedidos y stock.

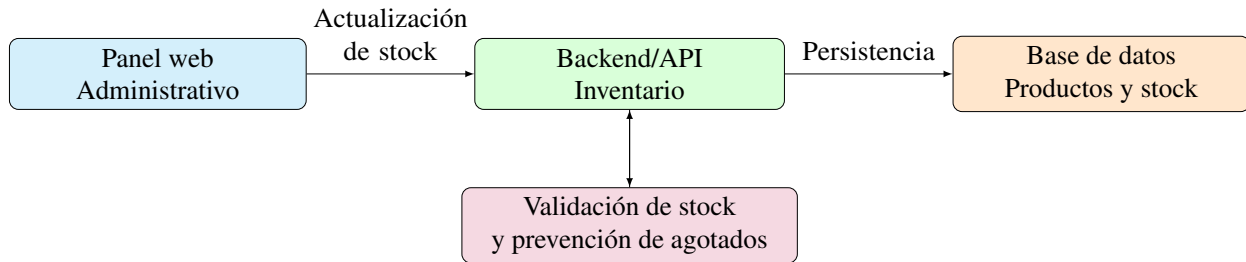


Figura 25. *Diagrama del módulo de inventario*

Diagrama del módulo de repartidores

El diagrama describe el proceso de registro, aprobación y asignación de pedidos a repartidores.

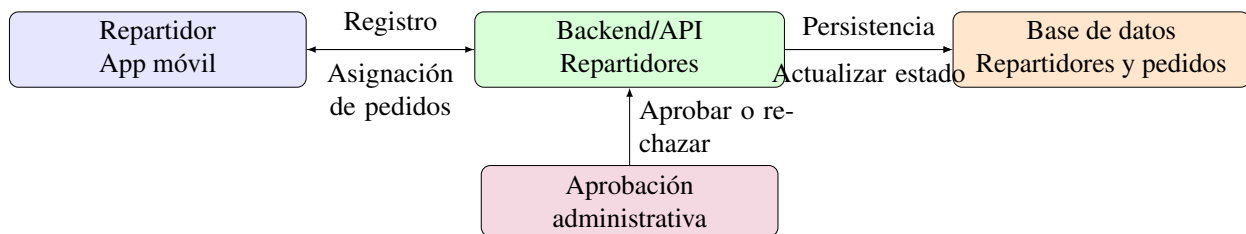


Figura 26. *Diagrama del módulo de repartidores*

Apéndice H

Arquitectura del despliegue

Se muestra la arquitectura de despliegue en staging con los servicios en VPS, panel web en Vercel y app móvil distribuida como APK.

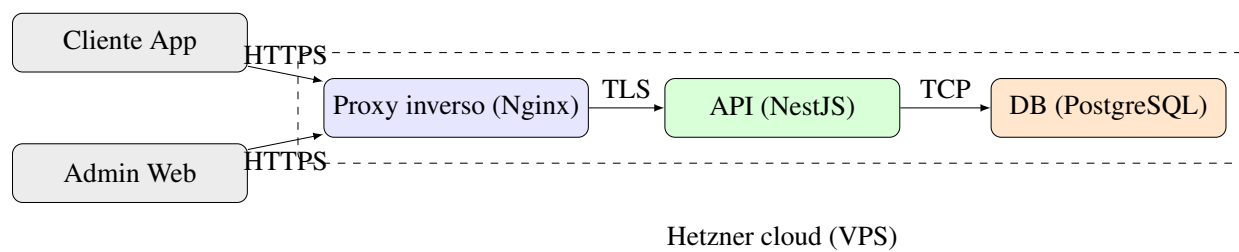


Figura 27. *Arquitectura del despliegue de la plataforma*

Apéndice I

Diagrama de red del sistema

El diagrama de red ilustra la arquitectura general del sistema y la interacción entre usuarios web y móviles, la API y servicios externos.

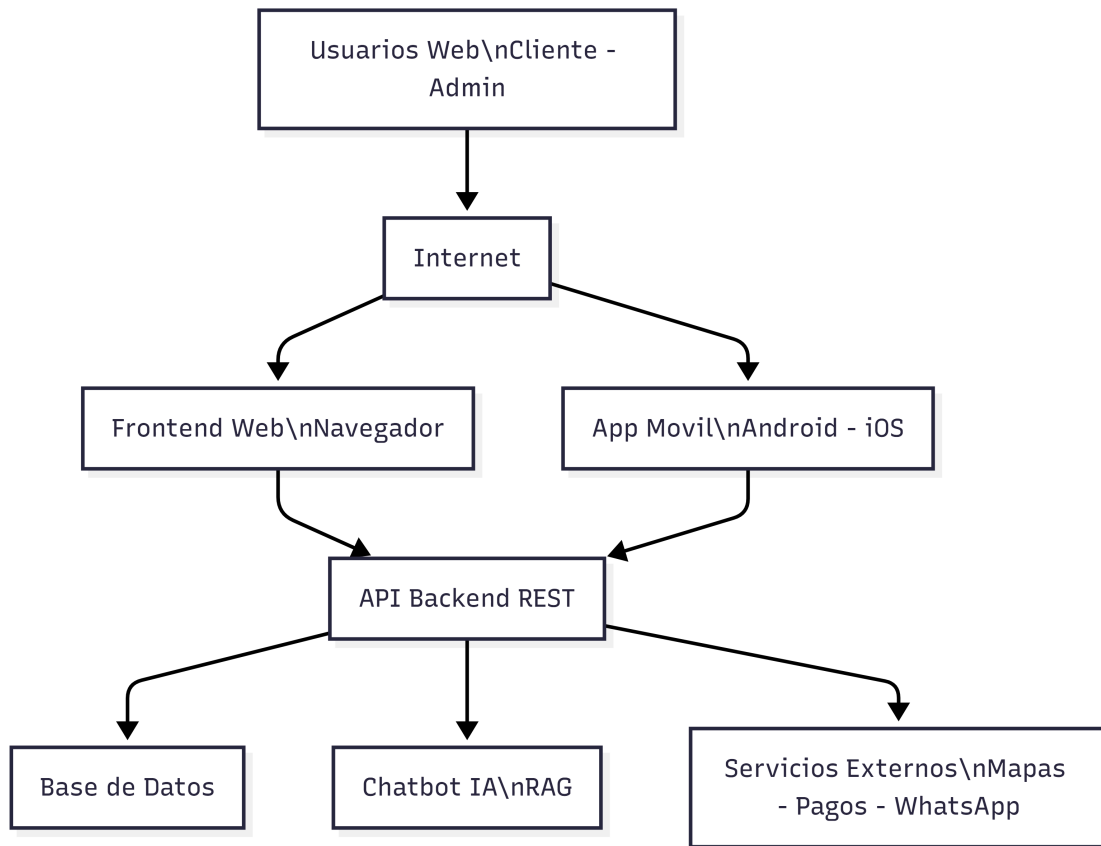


Figura 28. *Diagrama de red y arquitectura de la plataforma SaaS web y móvil*

Nota. El diagrama de red muestra la interacción entre el frontend web y móvil, la API backend, la base de datos, el chatbot con inteligencia artificial (por WhatsApp) y servicios externos como geolocalización; los medios de pago son efectivo y transferencia (sin pasarela de pagos).

Apéndice J

Procedimiento detallado de validación en staging

Este apéndice amplía el procedimiento de validación técnica descrito en el capítulo Método, con el detalle de los pasos ejecutados en entorno controlado.

1. Despliegue en entorno de staging

El backend se desplegó en un servidor VPS de Hetzner, utilizando Docker para contenerizar la aplicación y la base de datos PostgreSQL. Se configuraron ajustes de seguridad básicos (helmet, CORS, rate limiting) y variables de entorno mediante ConfigModule. La API se ejecutó en HTTPS y se activaron logs estructurados para monitoreo. El panel web se desplegó en Vercel y la aplicación móvil se distribuyó como APK para dispositivos de prueba.

2. Orientación a usuarios de prueba

Se realizó una sesión de orientación breve (aproximadamente 1 hora) con usuarios de la PYME piloto (administrador, personal de cocina y repartidores), explicando flujos principales y tareas de prueba.

3. Diseño de escenarios de prueba

Se definieron escenarios predefinidos para cubrir flujos críticos: (1) registro de pedido completo por cliente, (2) confirmación y preparación en cocina, (3) asignación y entrega por repartidor, (4) gestión de productos y categorías por administrador, (5) consulta al chatbot por WhatsApp. Cada

escenario utilizó datos simulados.

4. Ejecución de tareas guiadas

Los usuarios ejecutaron tareas bajo observación estructurada, registrando tiempos, errores y completitud de flujos. Las pruebas se realizaron en sesiones controladas durante un período acotado.

5. Recolección y análisis de datos

Se aplicó el cuestionario SUS al finalizar tareas guiadas y se consolidaron fichas de observación con métricas de desempeño. Se recopilaron logs para analizar disponibilidad, latencia y tasa de errores.

Tabla 7: Tareas evaluadas en pruebas de usabilidad

N.º	Tarea	Rol	Criterio de éxito
1	Crear un nuevo pedido	Cliente	Completar en < 3 min
2	Consultar estado de pedido	Cliente	Localizar en < 30 seg
3	Agregar producto al catálogo	Administrador	Completar sin errores
4	Asignar pedido a repartidor	Administrador	Completar en < 1 min
5	Marcar pedido como entregado	Repartidor	Completar sin ayuda
6	Realizar consulta al chatbot por WhatsApp	Cliente	Obtener respuesta pertinente